# Smart City Utilities & Infrastructure Management System

Discovery, architecture and 14-day delivery plan for Dhanush & Gagan

Plan v1 · 19 October 2026 · Phases 1–3 (no code written yet)


How to read this document
A. Executive summary
Phase 1 discovery: what the documents actually say
1.1 The official rubric (verified from `DBMS Level 3.pdf`)
1.2 Every proposed feature in the source document
1.3 Every entity, relationship and collection in the source
1.4 Inconsistencies and technical weaknesses found
1.5 Open questions for Dhanush (none block Day 1)
B. Rubric-to-feature traceability matrix
C. Architecture and technology decisions
C.1 Final technology stack
C.2 Is a separate Express backend necessary? No.
C.3 Architecture diagram
C.4 Database connection strategy
C.5 API conventions
C.6 Authentication and authorisation design
C.7 Environment variables (`.env.example`, placeholders only)
C.8 Local development setup (target, to be verified on Day 1)
C.9 Deployment strategy
D. ER and schema review, with corrections
D.1 Final table list (15 tables)
D.2 ER diagram (crow's-foot, text form; Mermaid source goes in `docs/er-diagram.md`)
D.3 Relational schema with all constraints (DDL to be written on Day 2)
D.4 Advanced SQL catalogue (rubric rows 6, 7, 8)
D.5 MongoDB design (rubric rows 4 and 9)
D.6 Cross-database integration
E. Feature backlog (P0 / P1 / P2)
P0: mandatory for the rubric (never cut)
P1: high-value product features
P2: stretch (only if ahead on Day 11)
F. The 14-day execution plan (Dhanush & Gagan)
G. Repository and file structure
H. Testing, security, Git and CI strategy
H.1 Testing pyramid
H.2 Security checklist
H.3 Git workflow (two people)
H.4 CI (GitHub Actions)
I. Risks, assumptions and contingencies
J. Final acceptance checklist (demo and viva)
J.1 The single best first milestone: M1 "Hybrid vertical slice" (end of Day 3)
J.2 Demo script (12 minutes, from `npm run demo:reset`)
J.3 Final checklist
J.4 Viva quick answers (starter set; full list in `docs/viva.md`)
Phase 3: what needs Dhanush's approval

## How to read this document

This is the complete proposal for the DBMS Level 3 mini-project. Every section has two layers:

> **In simple words** boxes explain the idea the way you would say it in the viva.

The text and tables around them carry the full technical detail: the exact tables, constraints, queries, APIs, tests and daily tasks.

**What was inspected.** Both source documents were read in full: `Pasted text.txt` (the earlier project write-up, schema and data flow) and `DBMS Level 3.pdf` (PES University, Aug–Dec 2026, Experiential Learning Level 3 mini-project rubric). No Git repository is attached to the project yet, so this plan treats the codebase as greenfield. A GitHub repo can be attached later and Day 1 creates it.

**Status labels used throughout.** *Official* means the rubric or source document requires it. *Recommended* means it is our engineering proposal. *Optional* means a stretch item. Nothing in this document claims anything is built: every rubric item starts as **Planned**.

---

## A. Executive summary

> **In simple words.** We are building a website for a pretend city's electricity, water and gas department. Fake smart meters send readings every few seconds into MongoDB. Once a month, the system adds up each meter's usage, stores that total in PostgreSQL, and PostgreSQL itself calculates the slab-based bill. Residents log in to see their bills and live usage, staff fix problems, and admins watch the whole city. If a pipe "bursts" in our simulator, the system notices the strange readings, explains why, and opens a repair ticket automatically. The point of the project is to show *why* money lives in SQL and sensor data lives in NoSQL, and how the two stay consistent.

**Problem statement (from the source, preserved).** Municipal utilities keep transactional data (residents, payments) separate from operational data (sensor readings, maintenance logs). This silo causes late leak detection, inaccurate billing and no real-time transparency for citizens, and existing systems cannot ingest the varied formats of heterogeneous IoT meters.

**Objectives (from the source, preserved).**

1. Hybrid database architecture: PostgreSQL for structured, ACID data and MongoDB for high-volume, flexible sensor and log data.
2. Real-time infrastructure monitoring of simulated electricity, water and gas meters on a live dashboard.
3. Automated anomaly detection that compares live NoSQL consumption with historical SQL baselines to flag leaks, spikes and outages.
4. Tiered billing computed by SQL stored procedures from consumption aggregated in NoSQL.
5. A unified citizen portal showing account data (SQL) beside live usage (NoSQL).

**What we will ship in 14 days.** One Next.js application (TypeScript) with three role-specific areas (Resident, Maintenance Staff, Municipal Admin), a PostgreSQL 17 database that enforces money rules and role-based access with Row-Level Security, a MongoDB 8 database with a time-series collection for readings, a deterministic IoT simulator, an explainable anomaly detector, a tiered billing engine, a maintenance-ticket workflow, a "Pipe Burst" disaster scenario, automated tests, GitHub Actions CI, and a viva pack.

**Our three opinions that shape everything.**

1. **The database does the important work, not the JavaScript.** Billing, payment state, ticket transitions and access control live in PostgreSQL functions, procedures, triggers and RLS policies. This is a DBMS evaluation, and the examiner must *see* the database working.
2. **One application, not two.** Next.js route handlers are the backend. A separate Express server would duplicate auth, validation and deployment for no marks.
3. **Fewer, finished workflows.** Ten demo steps that work every time beat thirty screens that half-work. Redis, weather APIs, email/SMS and maps are cut or deferred (see section I).

---

## Phase 1 discovery: what the documents actually say

### 1.1 The official rubric (verified from `DBMS Level 3.pdf`)

The PDF has two tables: a category summary and a 14-row evaluation table with partial-credit columns. The 14-row table is what examiners mark against, so this plan is organised around it.

| # | Evaluation component | Max | Partial | Category (marks) |
|---|---|---|---|---|
| 1 | ER diagram | 2 | 1 | Database Design (5) |
| 2 | Relational schema with keys, relationships, constraints | 3 | 2 | Database Design |
| 3 | SQL database implementation: DDL with constraints | 4 | 3 | Database Creation (9) |
| 4 | NoSQL/Vector database creation | 4 | 3 | Database Creation |
| 5 | Technology justification | 1 | 0 | Database Creation |
| 6 | DML operations | 4 | 3 | SQL Ops & Queries (8) |
| 7 | Nested / correlated / recursive queries, JOIN queries | 4 | 3 | SQL Ops & Queries |
| 8 | Advanced SQL: procedure/function, trigger, CTE | 6 | 3 | Advanced DB Concepts (13) |
| 9 | NoSQL/Vector feature, RUD | 7 | 4 | Advanced DB Concepts |

| # | Evaluation component | Max | Partial | Category (marks) |
|---|---|---|---|---|
| 10 | Next.js interface | 3 | 2 | Web App & Integration (5) |
| 11 | Backend/API and database integration | 2 | 1 | Web App & Integration |
| 12 | Authentication (frontend) + minimum two roles + RBAC **in the database** | 4 | 3 | AuthN & AuthZ (4) |
| 13 | Documentation, Git/GitHub | 2 | 1 | Docs, Git, Demo, Testing, Originality (6) |
| 14 | Originality, individual contribution, **Modification/Viva** | 4 | 2 | same |
| | **Total** | **50** | 31 | |

**Things the rubric says that the brief did not stress:**

- **Row 12 requires RBAC enforced in the database**, not just in the app. That makes PostgreSQL roles + Row-Level Security a P0 item.
- **Row 14 includes "Modification"**. Examiners commonly ask you to change something live during the viva (add a column, change a slab, add a filter). Both Dhanush and Gagan must be able to modify *any* part, so the plan includes cross-review and a modification drill (Day 13).
- **Row 9 says "RUD"** (Read, Update, Delete) on the NoSQL side and is worth 7 marks, the largest single row. Inserts alone are not enough: we need visible read, update and delete operations on MongoDB data.
- **Row 5 has no partial credit** (1 or 0). A written, specific technology justification is mandatory.
- **Next.js is named explicitly** (row 10). The source suggested React or Angular; Next.js is the official requirement and wins.
- "Vector" appears as an alternative to NoSQL. We satisfy the rows with MongoDB; a vector feature is an optional P2 item and is not needed for full marks.

## 1.2 Every proposed feature in the source document

| Source feature | Keep? | Notes |
|---|---|---|
| Tiered billing engine (SQL procedure on NoSQL aggregates) | Keep, P0 | Core of rows 8 and 11. |
| Anomaly and leak detection (NoSQL live vs SQL history) | Keep, P0 (rule-based) | Fix the unit mismatch, see 1.4. |
| Live infrastructure dashboard, zone-wise demand, drill-down | Keep, P1 | Periodic refresh (5 s polling), not WebSockets. |
| Resident self-service portal (profile, dues, live graph) | Keep, P0 | Main demo of the SQL + NoSQL merge. |
| Automated maintenance workflow (ticket on anomaly) | Keep, P0 | Ticket in SQL, notes/log in Mongo. |
| Email/SMS alerts to resident | Change | Replaced by in-app notifications. Real email/SMS is P2. |
| "What-if" disaster simulator, Map-Reduce for rising usage | Keep as Pipe Burst + Outage scenario, P1 | Map-Reduce is deprecated in MongoDB; use aggregation pipelines. |
| Transformer/substation cross-reference | Optional P2 | Not in the schema; a zone hierarchy gives the same "where is the problem" story. |
| City heatmap with clickable properties | Change to zone grid / choropleth-lite, P1 | A real map adds risk; a zone tile grid is enough. |
| Redis cache of last 5 minutes | Cut (P2) | Solves no demonstrated problem at our scale; Mongo index on (meter, ts) returns latest readings in milliseconds. |
| `weather_alerts` collection from an external API | Cut (P2) | No source defined, adds a flaky dependency, earns no rubric marks. |
| Generator of 10,000 readings/minute + 500 residents | Scale honestly | Target 300 meters at 1 reading/min live and ~90 days backfill (~3–4 M docs is possible but we will only claim what the load test measures). |

## 1.3 Every entity, relationship and collection in the source

**SQL entities (10):** RESIDENT, PROPERTY, UTILITY_METER, BILLING_CYCLE, UTILITY_RATE, METER_READING, INVOICE, INVOICE_LINE_ITEM, PAYMENT, MAINTENANCE_TICKET.

**Relationships:** Resident 1–M Property; Property 1–M Utility_Meter; Property 1–M Invoice; Property 1–M Maintenance_Ticket; Utility_Meter 1–M Meter_Reading; Billing_Cycle 1–M Invoice; Utility_Rate 1–M Invoice_Line_Item; Invoice 1–M Invoice_Line_Item; Invoice 1–1 Payment.

**Key constraints in the source:** UUID PKs; UNIQUE government_id, email; CHECK property_type, square_feet > 0, utility_type, tier_level IN (1,2,3), price_per_unit > 0, month 1–12, total_units_consumed ≥ 0, total_amount ≥ 0, payment_status, payment_method, issue_type, severity, status; UNIQUE (year, month) on cycle; UNIQUE (meter_id, cycle_id) on reading; UNIQUE invoice_id and transaction_id on payment; ON DELETE CASCADE on property, meter and payment FKs.

**SQL operations named in the source:** tiered-pricing stored procedure; JOIN query for residents with 3+ months overdue grouped by zone; UPDATE transaction with ROLLBACK on failed payment.

**MongoDB collections in the source:** `smart_meter_readings` / `smart_meter_stream` (two names for one thing), `maintenance_tickets`, `maintenance_audit_log`, `weather_alerts`. Indexes: `{meter_id:1, timestamp:-1}`, `{property_id:1}`.

## 1.4 Inconsistencies and technical weaknesses found

These are real problems in the supplied design. Each one either breaks correctness or costs marks.

| # | Problem in the source | Why it matters | Fix (rubric impact) |
|---|---|---|---|
| W1 | Monthly usage = `$sum: "$readings.kwh"`, but `kwh: 12450` is a **cumulative register** (the meter's odometer). | Summing odometer values gives absurd bills (millions of units). | Consumption = last register − first register in the window (or sum of deltas). Duplicate or out-of-order readings then cannot inflate a bill. Strengthens rows 9 and 11. |
| W2 | Anomaly rule compares a **24-hour average of hourly kWh** (Mongo) with **average monthly invoice units** (SQL). | Different units and windows; the 3× rule would fire constantly or never. | Compare like with like: same-hour-of-day baseline over the last 14–28 days, normalised per hour. SQL monthly history is used for the "unusual month" resident alert. |
| W3 | Tickets are stored in **SQL** in the schema but in **MongoDB** in the feature text. | Two sources of truth; nobody can say which is right in the viva. | Ticket header (state, assignee, severity, FK to property) in PostgreSQL. Free-form notes, sensor snapshots and attachment metadata in Mongo `maintenance_logs`. Clear boundary. |
| W4 | `PAYMENT` is strictly 1:1 with `INVOICE`. | Failed attempts, retries and replays cannot be recorded; "ROLLBACK on failed payment" has nowhere to log the failure. | `payment` holds every attempt with an `idempotency_key UNIQUE`; a partial unique index allows only **one SUCCEEDED** payment per invoice. Keeps the 1:1 "final payment" rule. |
| W5 | `UTILITY_RATE.max_units NOT NULL`, and "a rate cannot overlap" is only a comment. | Top slab cannot be open-ended; overlapping tariffs are possible, so bills become ambiguous. | `max_units` NULL = unbounded; `EXCLUDE USING gist` prevents overlapping validity per (utility, tier); a constraint trigger checks slabs are contiguous. |
| W6 | `INVOICE` has no UNIQUE (property_id, cycle_id). | The same bill can be generated twice (retries, double clicks). | Add the UNIQUE constraint and make the procedure idempotent. |
| W7 | `INVOICE_LINE_ITEM` has a single `rate_applied` and no FK to `UTILITY_RATE`, although the ER says Rate 1–M Line Item. | A tiered bill uses several rates; the applied tariff is not auditable. | One line per (utility, tier) with `rate_id` FK and snapshotted `unit_price`. |
| W8 | `METER_READING` cannot tell which cycle window it covers or where it came from. | No traceability back to Mongo; no protection against changing a reading after billing. | Add `opening_register`, `closing_register`, `source_doc_count`, `aggregation_run_id`, `is_estimated`; trigger blocks edits once invoiced. |
| W9 | `ON DELETE CASCADE` from resident → property → invoice → payment. | Deleting a person would silently erase financial history: illegal for a utility. | `RESTRICT` on financial paths; `CASCADE` only for dependent technical rows (e.g. ticket history). Soft-delete for residents. |
| W10 | `government_id` stored for real residents. | The brief forbids real ID numbers. | Synthetic `citizen_ref` like `SYN-000123`, clearly labelled synthetic. |
| W11 | Anomaly flow sets `properties.flag = 'UNDER_INVESTIGATION'`, but no such column exists. | Code would fail. | Add `property.service_status` with CHECK. |
| W12 | `zone_type` is free text. | Typos split analytics ("North" vs "north"). | `zone` table with a `parent_zone_id` hierarchy (City → District → Ward). Also gives a **meaningful recursive CTE** (row 7). |
| W13 | No users, roles, sessions or staff entity. | Rubric row 12 cannot be met. | `app_user`, `user_session`, staff linked to zones; PostgreSQL roles and RLS. |
| W14 | `billing_cycle` has no check that start < end ≤ due. | Invalid cycles. | CHECK constraints. |
| W15 | Overdue status needs time to pass; nothing sets it. | "3+ months overdue" query would return nothing. | `fn_mark_overdue()` run by the admin "close cycle" action and in seed data. |
| W16 | MongoDB Map-Reduce is proposed. | Deprecated since MongoDB 5.0. | Aggregation pipelines (`$setWindowFields`, `$dateTrunc`, `$group`). |
| W17 | `smart_meter_readings` vs `smart_meter_stream` naming. | Confusing docs. | One name: `meter_readings`. |
| W18 | "Millions of readings per minute" claims. | Cannot be proven on a laptop. | Report measured throughput only. |
| W19 | Penalty % exists but is never applied. | Dead column. | `fn_mark_overdue` adds a late-fee line item once per invoice (P1). |
| W20 | Payment method includes card types. | Implies real card handling. | Payments are **simulated** and labelled "Simulated gateway" in the UI. No card numbers are ever collected. |

## 1.5 Open questions for Dhanush (none block Day 1)

1. **Viva format:** will the demo run on your laptop, or must it be hosted? *Default: run locally with Docker Compose; optional cloud deploy on Day 12.*
2. **Currency and units:** *Default: ₹ (INR), electricity in kWh, water in kilolitres (kL), gas in m³ (SCM).*
3. **Does the guide want the 50/50 SQL–NoSQL split taken literally?** The source mentions it; the rubric weights SQL more heavily. *Default: SQL is the system of record; Mongo gets substantial, visible work (rows 4 and 9 = 11 marks).*
4. **Who presents which part?** *Default: the ownership split in section F; both can explain everything.*

## B. Rubric-to-feature traceability matrix

**In simple words.** For every mark in the rubric, this table says what we build, where the code will live, what test proves it, and what we click during the demo. Status is “Planned” everywhere because nothing is built yet; we will update it as each piece is proven.

Paths are relative to the repository (structure in section G). Test IDs refer to section H.

| Row | Rubric item (marks) | Feature that earns it | DB object / API | Source file(s) | Proving test | Demo step | Status |
|---|---|---|---|---|---|---|---|
| 1 | ER diagram (2) | ER diagram with crow’s-foot cardinalities for all 15 tables | — | `docs/er-diagram.md` (Mermaid) + exported PNG | Diagram regenerated from schema by `npm run docs:er`; reviewed in PR | Show diagram slide, trace Resident → Property → Meter → Reading → Invoice → Payment | Planned |
| 2 | Relational schema (3) | Normalised (3NF) schema with PK, FK, UNIQUE, CHECK, EXCLUDE | All tables | `docs/relational-schema.md` | `T-SQL-01` constraint catalogue test lists every constraint | Walk the schema page; point to keys and constraints | Planned |
| 3 | DDL with constraints (4) | Migrations creating tables, constraints, indexes, extensions | `0001_core.sql` … `0006_rls.sql` | `db/postgres/migrations/*.sql` | `T-SQL-02` invalid inserts rejected (bad month, negative units, overlapping tariff, duplicate invoice) | Run `psql` INSERTs live; show each rejection message | Planned |
| 4 | NoSQL creation (4) | Time-series collection `meter_readings`; validated collections `maintenance_logs`, `anomaly_events`, `notifications`, `ingestion_batches`; indexes; TTL | `createCollection` with `timeseries`, `$jsonSchema` validators | `db/mongo/setup.ts`, `db/mongo/schemas/*.json` | `T-MDB-01` collections and indexes exist; `T-MDB-02` invalid document rejected by validator | Mongo shell / Compass: `db.getCollectionInfos()`, insert a bad document | Planned |
| 5 | Technology justification (1) | Written comparison and reasons | — | `docs/technology-justification.md` | Doc review checklist | One slide: why Postgres, why Mongo, why not Redis | Planned |
| 6 | DML (4) | INSERT (residents, readings upsert), UPDATE (ticket status, payment), DELETE (tariff draft, resident soft-delete), `INSERT … ON CONFLICT`, `UPDATE … RETURNING`, `MERGE` for reading upsert | Tables + `sql/dml/*.sql` | `db/postgres/queries/dml.sql`, `src/server/repos/*.ts` | `T-SQL-03` DML suite | “SQL Lab” admin page runs each catalogued statement and shows rows affected | Planned |
| 7 | Nested / correlated / recursive + JOIN (4) | Q1 recursive zone roll-up; Q2 correlated “above zone average”; Q3 nested “3+ overdue”; Q4 NOT EXISTS “meters missing reading”; Q5–Q7 multi-table JOINs | `sql/queries/q*.sql`, views | `db/postgres/queries/*.sql` | `T-SQL-04` each query returns the expected rows on the fixed seed | SQL Lab page: run Q1–Q7, show SQL text + result | Planned |
| 8 | Procedure/function, trigger, CTE (6) | `fn_tiered_charge`, `sp_generate_invoice`, `sp_record_payment`, `fn_open_anomaly_ticket`, `fn_mark_overdue`; triggers T1–T5; analytical CTEs C1–C3 | `0003_functions.sql`, `0004_triggers.sql` | `db/postgres/migrations/` | `T-SQL-05..09` success + failure test for each object | Generate invoice → show line items; illegal ticket transition rejected; edit after invoicing rejected | Planned |
| 9 | NoSQL feature, RUD (7) | Read: aggregation pipelines (hourly, daily, zone, latest), time-range queries; Update: ticket notes `$push`, acknowledge anomaly `$set`, notification read; Delete: dismiss notification, purge demo dataset, TTL retention; bulk ingest | `meter_readings`, `maintenance_logs`, `anomaly_events`, `notifications` | `src/server/mongo/*.ts`, `db/mongo/pipelines/*.ts` | `T-MDB-03..08` (aggregations, malformed/out-of-order readings, RUD ops) | Live chart updates; add note to ticket; acknowledge alert; delete notification; show pipeline JSON | Planned |
| 10 | Next.js interface (3) | Role-aware app: Resident, Staff, Admin areas with charts, tables, filters | App Router pages | `src/app/(resident)`, `(staff)`, `(admin)` | `T-E2E-01..04` Playwright | Log in as each role, show distinct UI | Planned |
| 11 | Backend/API + DB integration (2) | Route handlers merging Postgres + Mongo; billing bridge; anomaly bridge | `/api/*` | `src/app/api/**/route.ts`, `src/server/**` | `T-API-*`, `T-X-01..03` cross-DB tests | Resident dashboard shows SQL bill + Mongo live usage on one screen | Planned |

| Row | Rubric item (marks) | Feature that earns it | DB object / API | Source file(s) | Proving test | Demo step | Status |
|---|---|---|---|---|---|---|---|
| 12 | AuthN + 2+ roles + RBAC in DB (4) | Login with Argon2id + DB sessions; 3 roles; PostgreSQL roles `resident_role`, `staff_role`, `admin_role`; RLS policies; GRANTs | `app_user`, `user_session`, `0006_rls.sql` | `src/server/auth/*`, `src/middleware.ts` | `T-AUTH-01..06`, `T-RLS-01..05` | Resident A cannot see B's invoice even with a hand-crafted URL; `psql` as `resident_role` returns 0 rows and "permission denied" on UPDATE | Planned |
| 13 | Docs + Git/GitHub (2) | README, docs folder, protected main, PRs, CI | GitHub Actions | `README.md`, `docs/`, `.github/workflows/ci.yml` | CI green on `main` (inspected) | Show PR history and CI run | Planned |
| 14 | Originality, contribution, modification/viva (4) | Contribution log, viva Q&A, modification drill | — | `docs/contributions.md`, `docs/viva.md` | Day 13 drill: each member makes 3 live changes in < 10 min | Answer examiner's modification request live | Planned |

**Demo-to-rubric map (the 10 proofs requested in the brief):**

| Demo proof | Rubric rows |
|---|---|
| 1. Resident sees only own data | 10, 12 |
| 2. IoT reading appears in MongoDB | 4, 9 |
| 3. Consumption analytics update | 9, 10, 11 |
| 4. Injected anomaly creates an actionable ticket | 8, 9, 11 |
| 5. Monthly usage becomes a valid SQL reading | 6, 11 |
| 6. Tiered billing gives the expected invoice | 8 |
| 7. Payment changes invoice state safely | 6, 8 |
| 8. Complex SQL returns meaningful results | 7, 8 |
| 9. Constraints reject invalid data | 2, 3 |
| 10. CI and tests show quality | 13 |

## C. Architecture and technology decisions

> **In simple words.** There is one website (Next.js). Its server side talks to two databases. PostgreSQL is the bank: it keeps people, properties, tariffs, bills and payments, and it refuses anything that breaks the rules. MongoDB is the sensor diary: it stores every meter reading and the messy notes around repairs. A small background program (the simulator) pretends to be thousands of meters writing into MongoDB. Once a month, a "bridge" adds up the diary and hands clean totals to the bank, which then prints the bills.

### C.1 Final technology stack

Versions below are the latest seen on the npm registry on 19 Oct 2026. Day 1 pins exact versions in `package.json` after a compatibility check against the Next.js 16 docs; if any combination fails, we drop to the previous minor version rather than fight it.

| Layer | Choice | Version (to pin) | Why this, and what we rejected |
|---|---|---|---|
| Web framework | **Next.js (App Router)** + React | next 16.x, react 19.x | Rubric row 10 names Next.js. Route handlers give us the API in the same codebase. |
| Language | TypeScript, strict mode | 5.9.x line unless Next 16 confirms TS 7 support | Type safety across UI, API and DB rows. |
| Styling / components | Tailwind CSS 4 + shadcn/ui (Radix primitives) | tailwindcss 4.x | Accessible primitives, consistent look, copy-in components (no heavy UI kit). |
| Charts | Recharts | 3.x | React-native API, enough for line/area/bar; D3 is too slow to build with in 14 days. |
| Validation | Zod, shared between client forms and API | zod 4.x | One schema per input; also validates readings before Mongo insert. |
| SQL database | **PostgreSQL 17** | Docker `postgres:17` | Procedures, triggers, CTEs, `EXCLUDE` constraints, RLS, `MERGE`, `NUMERIC` money. MySQL lacks RLS and exclusion constraints. |
| SQL driver | `pg` (node-postgres), **raw parameterised SQL**, no ORM | pg 8.x | The SQL *is* the deliverable; an ORM would hide it from the examiner. Parameterised queries stop SQL injection. |
| Migrations | Numbered plain `.sql` files + a ~50-line runner with a `schema_migrations` table | — | Every DDL statement is visible and reviewable. |
| NoSQL database | **MongoDB 8.0** (Docker `mongo:8`), official `mongodb` driver | mongodb 7.x driver | Time-series collections, aggregation pipelines, `$jsonSchema` validators. No Mongoose: validators live in the database, where they earn marks. |

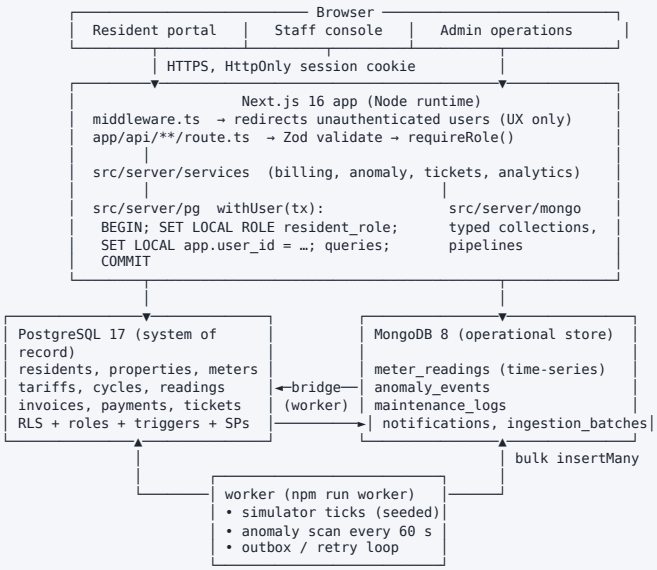
| Layer | Choice | Version (to pin) | Why this, and what we rejected |
|---|---|---|---|
| Auth | Custom: Argon2id password hashes + opaque session token in an HttpOnly cookie, session rows in Postgres | @node-rs/argon2 2.x | Small, explainable, revocable; the user id flows straight into RLS. Auth.js would add magic we must explain without adding marks. |
| Background worker | Node script run with `tsx` (`npm run worker`): simulator ticks, anomaly scan, outbox retry | tsx 4.x | Route handlers are request-scoped; long loops need a process. Same codebase, shared modules. |
| Tests | Vitest (unit, API, DB integration), Playwright (E2E) | vitest 5.x, @playwright/test 1.6x | Fast, TypeScript-native. |
| Tooling | ESLint, Prettier, `tsc --noEmit`, GitHub Actions, Docker Compose, Zod-validated env | — | |
| Cache | **No Redis** | — | Latest-reading query hits the `(meta.meter_id, ts)` index; measured on Day 7. Add Redis only if p95 > 200 ms. |

## C.2 Is a separate Express backend necessary? No.

| | Next.js route handlers only (chosen) | Next.js + separate Express API |
|---|---|---|
| Codebases / deploys | 1 | 2 |
| Auth & validation | Written once | Duplicated or shared via a package |
| Long-running jobs | Separate worker script (shares code) | Could live in Express |
| Viva explanation | “UI → route handler → service → repo → DB” | Extra network hop to justify |
| Risk in 14 days | Low | Medium |

The only thing Express would give us is a long-running process, and the worker script covers that without a second API.

## C.3 Architecture diagram




## C.4 Database connection strategy

- **PostgreSQL:** one `pg.Pool` per process, cached on `globalThis` so Next.js hot reload does not leak connections (pool size 10 app, 4 worker). The app logs in as `app_login`, a role that owns nothing and has no table rights of its own. Every request that touches user data runs inside `withUser(user, fn)`:

```
BEGIN;
SET LOCAL ROLE resident_role;                -- or staff_role / admin_role
SELECT set_config('app.user_id', $1, true);   -- true = transaction-local
-- ... queries; RLS policies read current_setting('app.user_id')
COMMIT;
```

  `SET LOCAL` disappears at COMMIT/ROLLBACK, so a pooled connection can never leak one user’s identity to the next request. Migrations run as `migrator`, which owns the tables; tables use `FORCE ROW LEVEL SECURITY` so even accidental owner queries are filtered.

- **MongoDB:** one `MongoClient` cached on `globalThis`, `maxPoolSize` 20. Two Mongo users: `ingest_writer` (insert into `meter_readings` and `ingestion_batches` only) used by the worker, and `app_rw` (read readings, CRUD on logs/notifications) used by the web app. This is least privilege on the NoSQL side too.
- **Timeouts:** `statement_timeout = 5s` for app roles; Mongo `maxTimeMS: 5000` on aggregations.

## C.5 API conventions

- REST-ish JSON under `/api/v1`. Nouns, plural: `/api/v1/invoices/:id`, `/api/v1/tickets/:id/transitions`.
- Every handler: `parse (Zod) → authenticate → authorise (role) → service → map errors`.
- Response envelope: `{ "data": … }` or `{ "error": { "code": "INVOICE_ALREADY_PAID", "message": "…", "details": … } }`. HTTP codes: 400 validation, 401 no session, 403 wrong role, 404 not found *or* not yours (we never reveal existence), 409 conflict/duplicate, 422 business rule, 500 unknown.
- Pagination: `?limit=20&cursor=<opaque>`; filters as query params (`utility`, `zone`, `from`, `to`, `severity`).
- Money is sent as **strings** (`"3900.00"`) to avoid floating-point loss; computed only in Postgres `NUMERIC(12,2)`.
- Mutations that can be retried (payments, invoice generation, simulator injection) accept an `Idempotency-Key` header.
- Postgres error codes are mapped: `23505` unique → 409, `23514` check → 422, `23P01` exclusion → 409, `42501` RLS/permission → 404/403.

## C.6 Authentication and authorisation design

> **In simple words.** There are three locks. The first lock is the website hiding pages you are not allowed to see (that is only convenience). The second lock is the server checking your role before doing anything (real protection). The third lock is PostgreSQL itself refusing to return another person's rows, even if our own code had a bug. The rubric gives marks specifically for the third lock.

| Layer | What it does | Bypassable? | Implemented by |
|---|---|---|---|
| Frontend route protection | Hides `/admin` from residents, redirects to `/login` | Yes, it is UX only | `middleware.ts`, role-aware nav |
| Backend authorisation | Every route handler checks session + role + ownership | Only by a bug in our code | `requireRole()`, services |
| Database access control | PostgreSQL roles + GRANTs + RLS filter rows by `app.user_id` | No, even raw SQL is filtered | `0006_rls.sql` |

- **Login:** email + password → Argon2id verify (memory 19 MiB, t=2, as per OWASP guidance) → 32-byte random token; store **SHA-256 of the token** in `user_session` with expiry (8 h, sliding) → cookie `sc_session`, `HttpOnly; Secure; SameSite=Lax; Path=/`.
- **Logout / revoke:** delete the session row. Admin can revoke all sessions for a user.
- **Brute force:** `app_user.failed_logins` + `locked_until` (5 failures → 15 min lock).
- **CSRF:** SameSite=Lax cookie + check `Origin` header on mutating requests.
- **Roles:** `resident`, `staff`, `admin` (three, rubric needs two).
- **RLS policy examples:**

```
        ALTER TABLE invoice ENABLE ROW LEVEL SECURITY;
        ALTER TABLE invoice FORCE ROW LEVEL SECURITY;

        CREATE POLICY invoice_resident_read ON invoice FOR SELECT TO resident_role
        USING (property_id IN (SELECT p.property_id FROM property p
               JOIN app_user u ON u.resident_id = p.resident_id
               WHERE u.user_id = current_setting('app.user_id')::uuid));

        CREATE POLICY invoice_admin_all ON invoice FOR ALL TO admin_role USING (true);

        GRANT SELECT ON invoice TO resident_role;          -- no INSERT/UPDATE/DELETE
        GRANT SELECT, INSERT, UPDATE ON invoice TO admin_role;
        GRANT EXECUTE ON PROCEDURE sp_record_payment TO resident_role;  -- pays own bill only
```

Staff see tickets assigned to them or in their zone; residents see only their own properties, meters, readings, invoices, line items, payments and tickets. Functions that must cross rows (e.g. invoice generation) are `SECURITY DEFINER` with a fixed `search_path` and an explicit role check inside.

## C.7 Environment variables (`.env.example`, placeholders only)

```
        # --- runtime ---
        NODE_ENV=development
        APP_URL=http://localhost:3000
        SESSION_SECRET=change-me-32-bytes-min          # used to sign CSRF/idempotency helpers
        # --- PostgreSQL ---
        DATABASE_URL=postgres://app_login:CHANGE_ME@localhost:5432/smartcity
        MIGRATOR_DATABASE_URL=postgres://migrator:CHANGE_ME@localhost:5432/smartcity
        TEST_DATABASE_URL=postgres://migrator:CHANGE_ME@localhost:5432/smartcity_test
        # --- MongoDB ---
        MONGODB_URI=mongodb://app_rw:CHANGE_ME@localhost:27017/smartcity?authSource=smartcity
        MONGODB_INGEST_URI=mongodb://ingest_writer:CHANGE_ME@localhost:27017/smartcity?authSource=smartcity
        MONGODB_TEST_DB=smartcity_test
        # --- simulator / anomaly ---
        SIM_SEED=42
        SIM_METER_COUNT=300
        SIM_INTERVAL_MS=5000
        ANOMALY_SPIKE_FACTOR=3
        ANOMALY_LEAK_MIN_FLOW_LPM=2
        ANOMALY_LEAK_WINDOW_MIN=120
        ANOMALY_MISSING_DATA_MIN=15
```

`src/env.ts` parses `process.env` with Zod at boot and refuses to start if a variable is missing or malformed. `.env` is in `.gitignore`; CI uses GitHub secrets only for the optional deploy.

## C.8 Local development setup (target, to be verified on Day 1)

```
        git clone <repo> && cd smart-city-utilities
        cp .env.example .env                # fill in local passwords
        docker compose up -d                # postgres:17 + mongo:8 with init scripts
        npm ci
```

```
npm run db:migrate                  # Postgres migrations + Mongo collections/indexes
npm run db:seed -- --seed 42        # synthetic city, 90 days backfill, 3 billing cycles
npm run dev                         # Next.js on :3000
npm run worker                      # simulator + anomaly scan (second terminal)
npm test                            # unit + integration
npm run test:e2e                    # Playwright
```

### C.9 Deployment strategy

- **Primary (viva):** the laptop runs Docker Compose with both databases and a production build (`npm run build && npm start`). It works without Wi-Fi, which is the biggest risk on viva day. A backup laptop (Gagan's) has the same setup by Day 12.
- **Optional (P2, Day 12 if green):** Vercel (web) + Neon Postgres + MongoDB Atlas free tier, worker on Render free tier. Check that the hosted Postgres allows `CREATE ROLE`/RLS before committing to it.
- **Demo data reset:** `npm run demo:reset` drops and reseeds both databases with seed 42 in under a minute, so every rehearsal starts identical.

---

## D. ER and schema review, with corrections

> **In simple words.** We keep the 10 tables from the original design and their names, fix the mistakes listed in 1.4, and add five small tables the original forgot: zones, users, sessions, ticket history and aggregation runs. Every fix exists to stop a wrong bill, a security hole or a confusing answer in the viva.

### D.1 Final table list (15 tables)

| Table | Status vs source | Purpose |
|---|---|---|
| `zone` | **New** | City → District → Ward hierarchy |
| `resident` | Kept, modified | Citizen (synthetic) |
| `property` | Kept, modified | Address, type, zone, service status |
| `utility_meter` | Kept, modified | Physical meter on a property |
| `billing_cycle` | Kept, modified | Monthly cycle |
| `utility_rate` | Kept, modified | Tariff slabs with validity |
| `aggregation_run` | **New** | Each Mongo→SQL bridge run, for idempotency and audit |
| `meter_reading` | Kept, modified | Monthly aggregated consumption (the bridge) |
| `invoice` | Kept, modified | Invoice header |
| `invoice_line_item` | Kept, modified | One line per utility × slab |
| `payment` | Kept, modified | Every payment attempt (simulated gateway) |
| `maintenance_ticket` | Kept, modified | Ticket header |
| `ticket_status_history` | **New** | Audit trail written by trigger |
| `app_user` | **New** | Login identity + role |
| `user_session` | **New** | Server-side sessions |

### D.2 ER diagram (crow's-foot, text form; Mermaid source goes in `docs/er-diagram.md`)

```
ZONE ──────────────< ZONE                    (parent → children, recursive)
ZONE ──────────────< PROPERTY
RESIDENT ──────────< PROPERTY                RESIDENT ──────○ APP_USER (0..1)
PROPERTY ──────────< UTILITY_METER ──────────< METER_READING >──────── BILLING_CYCLE
                                             METER_READING >──────── AGGREGATION_RUN
PROPERTY ──────────< INVOICE >──────────────────────────────────────── BILLING_CYCLE
INVOICE ──────────< INVOICE_LINE_ITEM >──── UTILITY_RATE
INVOICE ──────────< PAYMENT                  (attempts; at most one SUCCEEDED)
PROPERTY ──────────< MAINTENANCE_TICKET ────< TICKET_STATUS_HISTORY
APP_USER ──────────< MAINTENANCE_TICKET      (assigned_to)
APP_USER ──────────< USER_SESSION

Legend:  A ─────< B  means one A has many B;   A ──────○ B  means one A has zero or one B
```

Cardinalities: zone 1–M property; zone 1–M zone (parent); resident 1–M property; resident 1–0..1 app_user; property 1–M utility_meter; property 1–M invoice; property 1–M maintenance_ticket; utility_meter 1–M meter_reading; billing_cycle 1–M meter_reading; billing_cycle 1–M invoice; invoice 1–M invoice_line_item; utility_rate 1–M invoice_line_item; invoice 1–M payment (attempts) with at most one SUCCEEDED; maintenance_ticket 1–M ticket_status_history; app_user 1–M maintenance_ticket (assigned_to); app_user 1–M user_session; aggregation_run 1–M meter_reading.

### D.3 Relational schema with all constraints (DDL to be written on Day 2)

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;    -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS btree_gist;  -- EXCLUDE with = and &&

CREATE TYPE utility_kind AS ENUM ('ELECTRICITY','WATER','GAS');

CREATE TABLE zone (
  zone_id        SMALLSERIAL PRIMARY KEY,
  name           VARCHAR(60) NOT NULL UNIQUE,
  zone_level     VARCHAR(10) NOT NULL CHECK (zone_level IN ('CITY','DISTRICT','WARD')),
  parent_zone_id SMALLINT REFERENCES zone(zone_id) ON DELETE RESTRICT,
  CHECK ((zone_level = 'CITY') = (parent_zone_id IS NULL))
);

CREATE TABLE resident (
```

```
  resident_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  citizen_ref     VARCHAR(20) NOT NULL UNIQUE CHECK (citizen_ref LIKE 'SYN-%'),  -- synthetic, replaces government_id
  full_name       VARCHAR(100) NOT NULL,
  email           VARCHAR(254) NOT NULL UNIQUE CHECK (email = lower(email) AND position('@' in email) > 1),
  phone           VARCHAR(15) NOT NULL CHECK (phone ~ '^\+?[0-9]{10,14}$'),
  registered_date DATE NOT NULL DEFAULT CURRENT_DATE,
  is_active       BOOLEAN NOT NULL DEFAULT TRUE              -- soft delete
);

CREATE TABLE property (
  property_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  property_code  VARCHAR(12) NOT NULL UNIQUE,               -- human id, e.g. P-0055
  resident_id    UUID NOT NULL REFERENCES resident(resident_id) ON DELETE RESTRICT,
  zone_id        SMALLINT NOT NULL REFERENCES zone(zone_id) ON DELETE RESTRICT,
  address_line   VARCHAR(255) NOT NULL,
  city           VARCHAR(50)  NOT NULL DEFAULT 'Synthetic City',
  pincode        CHAR(6)      NOT NULL CHECK (pincode ~ '^[1-9][0-9]{5}$'),
  property_type  VARCHAR(12)  NOT NULL CHECK (property_type IN ('RESIDENTIAL','COMMERCIAL','INDUSTRIAL')),
  square_feet    INT          NOT NULL CHECK (square_feet BETWEEN 100 AND 1000000),
  service_status VARCHAR(20)  NOT NULL DEFAULT 'NORMAL'
                 CHECK (service_status IN ('NORMAL','UNDER_INVESTIGATION','DISCONNECTED'))
);
CREATE INDEX ON property (resident_id);
CREATE INDEX ON property (zone_id);

CREATE TABLE utility_meter (
  meter_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  meter_code        VARCHAR(16) NOT NULL UNIQUE,            -- e.g. MTR-E-000123 (used in Mongo)
  property_id       UUID NOT NULL REFERENCES property(property_id) ON DELETE RESTRICT,
  utility_type      utility_kind NOT NULL,
  installation_date DATE NOT NULL CHECK (installation_date <= CURRENT_DATE),
  meter_model       VARCHAR(50),
  is_active         BOOLEAN NOT NULL DEFAULT TRUE
);
-- one active meter per utility per property
CREATE UNIQUE INDEX uq_active_meter ON utility_meter (property_id, utility_type) WHERE is_active;

CREATE TABLE billing_cycle (
  cycle_id             SERIAL PRIMARY KEY,
  year                 SMALLINT NOT NULL CHECK (year BETWEEN 2020 AND 2100),
  month                SMALLINT NOT NULL CHECK (month BETWEEN 1 AND 12),
  cycle_start_date     DATE NOT NULL,
  cycle_end_date       DATE NOT NULL,
  due_date             DATE NOT NULL,
  late_penalty_percent NUMERIC(5,2) NOT NULL DEFAULT 5.00 CHECK (late_penalty_percent BETWEEN 0 AND 25),
  status               VARCHAR(10) NOT NULL DEFAULT 'OPEN' CHECK (status IN ('OPEN','BILLING','CLOSED')),
  UNIQUE (year, month),
  CHECK (cycle_start_date < cycle_end_date AND cycle_end_date < due_date)
);

CREATE TABLE utility_rate (
  rate_id              SERIAL PRIMARY KEY,
  utility_type         utility_kind NOT NULL,
  tier_level           SMALLINT NOT NULL CHECK (tier_level BETWEEN 1 AND 5),
  min_units            NUMERIC(12,3) NOT NULL CHECK (min_units >= 0),
  max_units            NUMERIC(12,3) CHECK (max_units IS NULL OR max_units > min_units),  -- NULL = no upper limit
  price_per_unit       NUMERIC(10,4) NOT NULL CHECK (price_per_unit > 0),
  effective_start_date DATE NOT NULL,
  effective_end_date   DATE CHECK (effective_end_date IS NULL OR effective_end_date > effective_start_date),
  -- no two versions of the same slab may be valid on the same day
  EXCLUDE USING gist (utility_type WITH =, tier_level WITH =,
                      daterange(effective_start_date, effective_end_date, '[)') WITH &&)
);
-- plus constraint trigger T4 (contiguous slabs, tier 1 starts at 0, only top tier unbounded)

CREATE TABLE aggregation_run (
  run_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  cycle_id      INT NOT NULL REFERENCES billing_cycle(cycle_id) ON DELETE RESTRICT,
  started_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  finished_at   TIMESTAMPTZ,
  status        VARCHAR(10) NOT NULL DEFAULT 'RUNNING' CHECK (status IN ('RUNNING','SUCCEEDED','FAILED')),
  meters_total  INT, meters_written INT, error TEXT
);
CREATE UNIQUE INDEX uq_one_running_run ON aggregation_run (cycle_id) WHERE status = 'RUNNING';

CREATE TABLE meter_reading (
  reading_id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  meter_id             UUID NOT NULL REFERENCES utility_meter(meter_id) ON DELETE RESTRICT,
  cycle_id             INT  NOT NULL REFERENCES billing_cycle(cycle_id) ON DELETE RESTRICT,
  run_id               UUID REFERENCES aggregation_run(run_id) ON DELETE SET NULL,
  opening_register     NUMERIC(15,3) NOT NULL CHECK (opening_register >= 0),
  closing_register     NUMERIC(15,3) NOT NULL,
  total_units_consumed NUMERIC(15,3) GENERATED ALWAYS AS (closing_register - opening_register) STORED,
  source_doc_count     INT NOT NULL CHECK (source_doc_count >= 0),
  is_estimated         BOOLEAN NOT NULL DEFAULT FALSE,      -- true when data gaps were filled
  reading_timestamp    TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (meter_id, cycle_id),
  CHECK (closing_register >= opening_register)
);

CREATE TABLE invoice (
  invoice_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_no     VARCHAR(20) NOT NULL UNIQUE,               -- INV-2026-09-000123
  property_id    UUID NOT NULL REFERENCES property(property_id) ON DELETE RESTRICT,
  cycle_id       INT  NOT NULL REFERENCES billing_cycle(cycle_id) ON DELETE RESTRICT,
  invoice_date   DATE NOT NULL DEFAULT CURRENT_DATE,
  total_amount   NUMERIC(12,2) NOT NULL DEFAULT 0 CHECK (total_amount >= 0),
  payment_status VARCHAR(10) NOT NULL DEFAULT 'PENDING'
                 CHECK (payment_status IN ('PENDING','PAID','OVERDUE','DISPUTED')),
  paid_at        TIMESTAMPTZ,
  UNIQUE (property_id, cycle_id),                           -- no duplicate bills
  CHECK ((payment_status = 'PAID') = (paid_at IS NOT NULL))
);
CREATE INDEX ON invoice (payment_status, cycle_id);

CREATE TABLE invoice_line_item (
  line_item_id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id     UUID NOT NULL REFERENCES invoice(invoice_id) ON DELETE CASCADE,
  utility_type   utility_kind NOT NULL,
  line_kind      VARCHAR(10) NOT NULL DEFAULT 'USAGE' CHECK (line_kind IN ('USAGE','LATE_FEE')),
  rate_id        INT REFERENCES utility_rate(rate_id) ON DELETE RESTRICT,
  tier_level     SMALLINT,
```

```
  units_consumed NUMERIC(15,3) NOT NULL CHECK (units_consumed >= 0),
  rate_applied   NUMERIC(10,4) NOT NULL CHECK (rate_applied >= 0),     -- price snapshot
  line_total     NUMERIC(12,2) NOT NULL CHECK (line_total >= 0),
  UNIQUE (invoice_id, utility_type, line_kind, tier_level),
  CHECK (line_kind = 'LATE_FEE' OR (rate_id IS NOT NULL AND tier_level IS NOT NULL))
);

CREATE TABLE payment (
  payment_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  invoice_id      UUID NOT NULL REFERENCES invoice(invoice_id) ON DELETE RESTRICT,
  idempotency_key VARCHAR(64) NOT NULL UNIQUE,              -- replayed requests hit this
  amount_paid     NUMERIC(12,2) NOT NULL CHECK (amount_paid > 0),
  payment_method  VARCHAR(12) NOT NULL CHECK (payment_method IN ('UPI_SIM','CARD_SIM','NETBANK_SIM','CASH')),
  status          VARCHAR(10) NOT NULL DEFAULT 'INITIATED' CHECK (status IN ('INITIATED','SUCCEEDED','FAILED')),
  transaction_id  VARCHAR(100) UNIQUE,                      -- simulated gateway reference
  failure_reason  TEXT,
  payment_date    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE UNIQUE INDEX uq_one_success_per_invoice ON payment (invoice_id) WHERE status = 'SUCCEEDED';

CREATE TABLE app_user (
  user_id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email         VARCHAR(254) NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,                              -- Argon2id encoded string
  role          VARCHAR(10) NOT NULL CHECK (role IN ('RESIDENT','STAFF','ADMIN')),
  resident_id   UUID UNIQUE REFERENCES resident(resident_id) ON DELETE RESTRICT,
  staff_zone_id SMALLINT REFERENCES zone(zone_id),
  failed_logins SMALLINT NOT NULL DEFAULT 0, locked_until TIMESTAMPTZ,
  CHECK ((role = 'RESIDENT') = (resident_id IS NOT NULL)),
  CHECK (role <> 'STAFF' OR staff_zone_id IS NOT NULL)
);

CREATE TABLE user_session (
  token_hash  BYTEA PRIMARY KEY,                            -- sha256(token); raw token only in cookie
  user_id     UUID NOT NULL REFERENCES app_user(user_id) ON DELETE CASCADE,
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),
  expires_at  TIMESTAMPTZ NOT NULL,
  CHECK (expires_at > created_at)
);

CREATE TABLE maintenance_ticket (
  ticket_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  ticket_no          VARCHAR(12) NOT NULL UNIQUE,           -- MT-000221
  property_id        UUID NOT NULL REFERENCES property(property_id) ON DELETE RESTRICT,
  meter_id           UUID REFERENCES utility_meter(meter_id) ON DELETE RESTRICT,
  source             VARCHAR(10) NOT NULL CHECK (source IN ('ANOMALY','RESIDENT','STAFF')),
  anomaly_key        VARCHAR(80),                           -- e.g. LEAK:MTR-W-000012
  issue_type         VARCHAR(20) NOT NULL CHECK (issue_type IN
                       ('LEAK','POWER_OUTAGE','VOLTAGE_SPIKE','METER_MALFUNCTION','LOW_PRESSURE','BILLING','OTHER')),
  severity           VARCHAR(8) NOT NULL CHECK (severity IN ('LOW','MEDIUM','HIGH','CRITICAL')),
  description        TEXT NOT NULL CHECK (length(description) BETWEEN 5 AND 2000),
  status             VARCHAR(12) NOT NULL DEFAULT 'OPEN'
                     CHECK (status IN ('OPEN','ASSIGNED','IN_PROGRESS','RESOLVED','CLOSED')),
  assigned_to        UUID REFERENCES app_user(user_id) ON DELETE SET NULL,
  reported_timestamp TIMESTAMPTZ NOT NULL DEFAULT now(),
  last_seen_at       TIMESTAMPTZ NOT NULL DEFAULT now(),    -- updated while anomaly continues
  occurrence_count   INT NOT NULL DEFAULT 1 CHECK (occurrence_count >= 1),
  resolved_at        TIMESTAMPTZ,
  resolution_notes   TEXT,
  CHECK (status NOT IN ('RESOLVED','CLOSED') OR (resolved_at IS NOT NULL AND resolution_notes IS NOT NULL))
);
-- the duplicate-ticket guard: one *active* ticket per continuing anomaly
CREATE UNIQUE INDEX uq_active_anomaly_ticket ON maintenance_ticket (anomaly_key)
  WHERE status IN ('OPEN','ASSIGNED','IN_PROGRESS');
CREATE INDEX ON maintenance_ticket (status, severity);
CREATE INDEX ON maintenance_ticket (assigned_to) WHERE status IN ('ASSIGNED','IN_PROGRESS');

CREATE TABLE ticket_status_history (
  history_id  BIGSERIAL PRIMARY KEY,
  ticket_id   UUID NOT NULL REFERENCES maintenance_ticket(ticket_id) ON DELETE CASCADE,
  from_status VARCHAR(12), to_status VARCHAR(12) NOT NULL,
  changed_by  UUID REFERENCES app_user(user_id),
  changed_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

**Normalisation note (viva).** Every non-key column depends on the whole key and nothing but the key (3NF). The one deliberate denormalisation is `invoice.total_amount` and `invoice_line_item.rate_applied`, which are **snapshots**: a bill must keep the price that applied on the day it was issued even if the tariff changes later. `total_amount` is kept consistent with the line items by trigger T3.

**ON DELETE rules (why).** Money paths use `RESTRICT` because financial history must never vanish. `invoice_line_item` uses `CASCADE` because a line cannot exist without its invoice (and invoices themselves can't be deleted once paid). `user_session` cascades with its user. `assigned_to` uses `SET NULL` so removing a staff account leaves the ticket.

## D.4 Advanced SQL catalogue (rubric rows 6, 7, 8)

Each object is listed with its business purpose, when it runs, its success and failure tests, and the one-line viva answer.

### Functions and procedures

| Object | Business purpose | Runs when | Success test | Failure / edge test | Viva line |
|---|---|---|---|---|---|
| `fn_tiered_charge(utility, units, as_of) RETURNS TABLE(tier, rate_id, units, price, amount)` | Split consumption across slabs valid on `as_of`. | Called by `sp_generate_invoice`; also by the "bill estimator" page. | 450 kWh with slabs 0–100 @5, 100–300 @8, 300+ @12 → 100×5 + 200×8 + 150×12 = **₹3,900.00**. | 0 units → zero rows/₹0; exactly 100 units → only tier 1; no tariff valid on date → raises `NO_TARIFF`. | "It walks the slabs in order and takes `LEAST(units, max) − min` for each." |

| Object | Business purpose | Runs when | Success test | Failure / edge test | Viva line |
|---|---|---|---|---|---|
| `sp_generate_invoice(property_id, cycle_id)` PROCEDURE | Create one invoice with lines for every utility in one transaction. | Admin "Generate bills for cycle" (loops properties) or per-property retry. | Invoice + lines inserted; total = sum of lines. | Run twice → second call returns the existing invoice (no duplicate, `ON CONFLICT DO NOTHING` + lock); missing `meter_reading` → whole procedure rolls back with `MISSING_READING`; two concurrent calls → one waits on `pg_advisory_xact_lock(hash(property, cycle))`. | "It is atomic: either the full bill exists or nothing does." |
| `sp_record_payment(invoice_id, amount, method, idem_key, simulate_outcome)` PROCEDURE | Record a (simulated) payment safely. | Resident clicks Pay. | SUCCEEDED → invoice PAID via trigger T5. | Same `idem_key` again → returns first result, no second row; amount ≠ outstanding → `AMOUNT_MISMATCH`; invoice already PAID → `ALREADY_PAID`; `simulate_outcome = FAIL` → attempt stored as FAILED, invoice stays PENDING (savepoint + rollback demo). | "The idempotency key makes a double-click harmless." |
| `fn_open_anomaly_ticket(property, meter, key, type, severity, description)` | Open a ticket for a new anomaly, or bump `last_seen_at` / `occurrence_count` on the active one. | Worker after each anomaly scan. | First call inserts; property → UNDER_INVESTIGATION. | Second call with the same key → updates the same ticket (`INSERT … ON CONFLICT (anomaly_key) WHERE status IN (...) DO UPDATE`). | "The partial unique index means one live ticket per problem." |
| `fn_mark_overdue(as_of)` | Mark unpaid invoices past due, add the late-fee line once. | Admin "Close cycle", seed script. | Pending past due → OVERDUE + LATE_FEE line. | Running twice adds no second fee (unique line key). | |

## Triggers (each with a real business job)

| Trigger | Table / timing | Purpose | Failure test |
|---|---|---|---|
| T1 `trg_ticket_transition` | `maintenance_ticket` BEFORE UPDATE OF status | Allow only OPEN→ASSIGNED→IN_PROGRESS→RESOLVED→CLOSED (plus RESOLVED→IN_PROGRESS reopen); set `resolved_at`; require `assigned_to` for ASSIGNED. | OPEN→CLOSED raises `ILLEGAL_TRANSITION`. |
| T2 `trg_ticket_history` | `maintenance_ticket` AFTER INSERT/UPDATE OF status | Append to `ticket_status_history` with `current_setting('app.user_id')`. | History row count equals transitions. |
| T3 `trg_invoice_total` | `invoice_line_item` AFTER INSERT/UPDATE/DELETE | Keep `invoice.total_amount = SUM(line_total)`; block line changes once invoice is PAID. | Editing a line of a PAID invoice raises `INVOICE_LOCKED`. |
| T4 `trg_tariff_contiguity` | `utility_rate` CONSTRAINT TRIGGER, DEFERRABLE INITIALLY DEFERRED | Slabs per utility/period must start at 0, have no gaps, and only the top tier may be unbounded. | Inserting a 0–100 and 150–300 pair fails at COMMIT with `TARIFF_GAP`. |
| T5 `trg_payment_settles_invoice` | `payment` AFTER INSERT/UPDATE OF status | On SUCCEEDED set invoice PAID + `paid_at`. | Second SUCCEEDED for same invoice blocked by partial unique index. |
| T6 `trg_reading_locked` | `meter_reading` BEFORE UPDATE/DELETE | Block changes once an invoice exists for that meter's property and cycle. | Re-aggregation after billing fails with `READING_LOCKED`. |

## Queries (rubric row 7) and CTEs (row 8)

| ID | Kind | Question it answers |
|---|---|---|
| Q1 | **Recursive CTE** | Total consumption and outstanding balance for any zone *including all its sub-zones* (City → Districts → Wards). |
| Q2 | **Correlated subquery** | Properties whose consumption this cycle is > 1.5× the average of other properties of the same type in the same zone. |
| Q3 | **Nested subquery + GROUP BY/HAVING + JOIN** (from the source) | Residents with 3+ overdue invoices, grouped by zone, for disconnection notices. |
| Q4 | **NOT EXISTS** | Active meters with no `meter_reading` for the current cycle (aggregation gaps). |
| Q5 | **5-table JOIN** | Resident statement: resident → property → invoice → line items → rate. |
| Q6 | **LEFT JOIN + aggregates** | Zone-wise outstanding and overdue amounts, including zones with zero invoices. |
| Q7 | **JOIN with staff** | Ticket backlog per staff member with average time-to-resolve. |
| C1 | **Analytical CTE + window** | Month-over-month change per property (`LAG`), top 10 increases. |
| C2 | **CTE** | Receivables ageing buckets (0–30, 31–60, 61–90, 90+ days). |
| C3 | **CTE** | Revenue by utility and tier: how much of revenue comes from the top slab. |

```
-- Q1: recursive zone roll-up. Readings and invoices are aggregated separately
-- before joining, so one invoice is never counted once per meter.
```

```sql
WITH RECURSIVE subtree AS (
  SELECT zone_id FROM zone WHERE zone_id = $1          -- anchor: the chosen zone
  UNION ALL
  SELECT z.zone_id FROM zone z JOIN subtree s ON z.parent_zone_id = s.zone_id
),
usage AS (
  SELECT um.utility_type, SUM(mr.total_units_consumed) AS units
  FROM subtree s
  JOIN property p       ON p.zone_id = s.zone_id
  JOIN utility_meter um ON um.property_id = p.property_id
  JOIN meter_reading mr ON mr.meter_id = um.meter_id AND mr.cycle_id = $2
  GROUP BY um.utility_type
),
dues AS (
  SELECT SUM(i.total_amount) AS outstanding
  FROM subtree s
  JOIN property p ON p.zone_id = s.zone_id
  JOIN invoice i  ON i.property_id = p.property_id
  WHERE i.payment_status IN ('PENDING','OVERDUE')
)
SELECT u.utility_type, u.units, d.outstanding FROM usage u CROSS JOIN dues d;
```

*Every query in this table gets a test with hand-checked expected rows on seed 42 (`T-SQL-04`).*

### DML catalogue (rubric row 6)

INSERT (seed residents, tickets), multi-row INSERT, `INSERT … ON CONFLICT DO UPDATE` (reading upsert), `MERGE` (alternative upsert, shown in SQL Lab), `UPDATE … RETURNING` (ticket assignment), `UPDATE … FROM` (overdue marking), `DELETE` (draft tariff not yet used; expired sessions), soft delete (resident `is_active`), and an explicit transaction with `SAVEPOINT`/`ROLLBACK TO` (failed payment).

## D.5 MongoDB design (rubric rows 4 and 9)

> **In simple words.** MongoDB gets the jobs that are big, fast and messy: millions of meter readings with different fields per meter type, and repair notes that grow over time. We still give it rules: required fields, indexes for every query we run, and an automatic clean-up of old raw data.

| Collection | Type | Shape (key fields) | Indexes | Access patterns |
|---|---|---|---|---|
| `meter_readings` | **Time-series** (`timeField: ts`, `metaField: meta`, `granularity: "minutes"`), `expireAfterSeconds` = 180 days | `{ ts, meta:{meter_code, property_code, utility, zone}, reading_id, seq, register, ...type-specific }` Electricity: `kwh_total, voltage, current, power_factor`; Water: `volume_l_total, flow_lpm, pressure_psi`; Gas: `m3_total, pressure_kpa`. Optional `fault:"SENSOR_STUCK"` | Auto bucket index; `{ "meta.meter_code":1, ts:-1 }`; `{ "meta.zone":1, ts:-1 }` | Latest N for a meter; hourly/daily series; zone totals; cycle window first/last register |
| `readings_daily` | Regular, materialised by `$merge` | `{ _id:{meter_code, day}, utility, zone, units, min_reg, max_reg, samples }` | `_id`, `{zone:1, "_id.day":1}` | Fast 90-day charts; the archive that survives TTL |
| `anomaly_events` | Regular + `$jsonSchema` | `{ _id: "<type>:<meter>:<window>", type, meter_code, property_code, severity, detected_at, window, observed, baseline, factor, reason, status: OPEN/ACK/RESOLVED, ticket_no, synced_to_sql: bool }` | `{status:1, detected_at:-1}`, `{synced_to_sql:1}` | Admin alert feed; acknowledge (**update**); outbox retry |
| `maintenance_logs` | Regular + `$jsonSchema` | `{ ticket_no, entries:[{at, by, kind:"NOTE/STATUS/SENSOR_SNAPSHOT", text, data}], attachments:[{name, mime, size}] }` | `{ticket_no:1}` unique | Timeline (**read**), add note `$push` (**update**), delete own note `$pull` (**delete**) |
| `notifications` | Regular + `$jsonSchema`, TTL 30 days | `{ user_id, kind, title, body, created_at, read_at }` | `{user_id:1, created_at:-1}` | Resident alerts; mark read (**update**); dismiss (**delete**) |
| `ingestion_batches` | Regular | `{ _id: batch_id, seed, meters, count, from, to, status }` | `_id` | Idempotent simulator batches; reset demo |

**Why cumulative registers (the fix for W1).** Monthly units = `max(register) − min(register)` inside the window (plus the last register before the window as opening). A duplicated or late reading has a register value already inside that range, so it cannot change the bill. Missing readings only lose resolution, not money. A register that goes *down* (meter replaced or faulty) is flagged as a `METER_MALFUNCTION` anomaly and the reading is marked `is_estimated`.

**Validation.** Time-series collections have limited validator support, so readings are validated twice: by Zod at the ingestion API and by a `$jsonSchema` validator where the server version accepts it (verified on Day 3; if not accepted, documented as a known limitation). All other collections use `$jsonSchema` with `validationAction: "error"`.

**Bulk ingestion.** The simulator buffers readings and calls `insertMany(batch, { ordered: false })` every tick. Batches carry a `batch_id`; re-running a batch with the same seed and time window is detected in `ingestion_batches` and skipped.

**Retention / archival.** Raw readings expire after 180 days (TTL); before that, a nightly `$merge` writes per-meter daily rollups to `readings_daily`, which is kept. Billing never depends on raw data older than the open cycle, because the monthly total is already in PostgreSQL.

**Aggregation pipelines to implement:**

1. *Hourly series for a meter:* `$match` meta + ts range → `$group` by `$dateTrunc: {unit: "hour"}` with `$min/$max` register → `$project` units = max − min.
2. *Daily zone consumption:* `$match` ts range → `$group` by zone, utility, day.
3. *Cycle totals for the bridge:* `$match` cycle window → `$group` by meter: first/last register (`$top`/`$bottom` sorted by ts), count, gaps.
4. *Baseline for anomaly:* `$setWindowFields` partitioned by meter, sorted by hour, rolling 14-day average for the same hour of day.
5. *Meter health:* `$group` by meter → last `ts`; compared with now to show “last seen” and offline meters.

**Security on the Mongo side.** Never pass request bodies directly into a filter; build filters from Zod-parsed primitives so `{"$gt": ""}` operator injection is impossible. Separate Mongo users for ingest and app (C.4).

### D.6 Cross-database integration

> **In simple words.** PostgreSQL and MongoDB can't share one transaction, so we never pretend they do. Instead, every hand-off between them is written so that running it twice does no harm, and a "to-do list" field tells the system what still needs to be copied across if something crashed half-way.

| Flow | Direction | Mechanism | Idempotency / recovery |
|---|---|---|---|
| Monthly usage → billing | Mongo → Postgres | Worker/admin starts an `aggregation_run`, runs pipeline 3, upserts `meter_reading` per meter in batches, marks run SUCCEEDED | `UNIQUE(meter_id, cycle_id)` + `ON CONFLICT DO UPDATE` while not invoiced; T6 locks after billing; a failed run is simply re-run |
| Anomaly → ticket | Mongo → Postgres | Detector upserts `anomaly_events` with deterministic `_id` and `synced_to_sql:false` (this is the **outbox**); then calls `fn_open_anomaly_ticket`; then sets `synced_to_sql:true, ticket_no` | Crash between steps leaves `synced_to_sql:false`; the worker retries every 60 s; the SQL function is itself idempotent via the partial unique index |
| Ticket status → log | Postgres → Mongo | After a successful SQL transition, append a STATUS entry to `maintenance_logs` | SQL history is authoritative; the reconciler rebuilds missing Mongo entries from `ticket_status_history` |
| Dashboard | Both → UI | `Promise.all` of a Postgres query (bill) and a Mongo query (live usage); merged JSON | Each half fails independently; UI shows partial data with an error badge |
| Reconciliation | Both | `npm run reconcile` + admin page: for the cycle, compare Mongo pipeline totals with `meter_reading`, list differences, unsynced events, and orphan meter codes | Test `T-X-03` |

---

## E. Feature backlog (P0 / P1 / P2)

> **In simple words.** P0 is what we must have to pass every rubric row. P1 makes the product impressive. P2 is only if we are ahead. If time runs out, we cut from the bottom of P2 upwards and never touch P0.

### P0: mandatory for the rubric (never cut)

| ID | Item | Rubric | Owner |
|---|---|---|---|
| P0-1 | Repo, CI, Docker Compose, env validation | 13 | Gagan |
| P0-2 | Full Postgres DDL with constraints, indexes, extensions | 2, 3 | Dhanush |
| P0-3 | ER diagram + relational schema docs | 1, 2 | Dhanush |
| P0-4 | Mongo collections, validators, indexes, TTL | 4 | Gagan |
| P0-5 | Deterministic seed (zones, 60 residents, 80 properties, ~240 meters, 3 cycles, tariffs, users) | 6 | Dhanush |
| P0-6 | Simulator: backfill + live ticks + bulk insert | 9 | Gagan |
| P0-7 | Auth: login/logout, Argon2id, sessions, 3 roles | 12 | Dhanush |
| P0-8 | DB roles, GRANTs, RLS policies + tests | 12 | Dhanush |
| P0-9 | Billing: `fn_tiered_charge`, `sp_generate_invoice`, bridge from Mongo | 8, 11 | Dhanush (SQL) + Gagan (pipeline) |
| P0-10 | Payments: `sp_record_payment`, idempotency, simulated gateway | 6, 8 | Dhanush |
| P0-11 | Triggers T1–T6 | 8 | Dhanush |
| P0-12 | Queries Q1–Q7, CTEs C1–C3, SQL Lab page | 7, 8 | Dhanush (SQL) + Gagan (page) |
| P0-13 | Mongo RUD: ticket notes, anomaly acknowledge, notifications | 9 | Gagan |
| P0-14 | Aggregations: hourly, daily, zone, meter health | 9 | Gagan |
| P0-15 | Resident portal: dashboard (SQL+Mongo), invoices, pay, tickets | 10, 11 | Gagan (UI) |

| ID | Item | Rubric | Owner |
|---|---|---|---|
| P0-16 | Admin dashboard: city summary, zone analytics, billing run, outstanding | 10 | Gagan |
| P0-17 | Staff console: assigned tickets, transitions, notes | 10 | Gagan |
| P0-18 | Anomaly detection: spike, leak, missing data; ticket creation | 9, 11 | Gagan (detector) + Dhanush (SQL fn) |
| P0-19 | Tests (unit, DB, API, RLS) in CI | 13 | both |
| P0-20 | README, docs, technology justification, viva pack, contribution log | 5, 13, 14 | both |

### P1: high-value product features

| ID | Item |
|---|---|
| P1-1 | Pipe Burst + Local Outage disaster scenario page with affected properties and live ticket creation |
| P1-2 | Duplicate-ticket suppression shown in the UI (occurrence count, last seen) |
| P1-3 | Drill-down: city → zone → property detail |
| P1-4 | Filters (utility, zone, time range, severity), pagination, sorting |
| P1-5 | Late fee via `fn_mark_overdue`; overdue banner for residents |
| P1-6 | Resident alerts: unusual month, due-date reminder (in-app) |
| P1-7 | Reconciliation page and `npm run reconcile` |
| P1-8 | Playwright E2E for 4 critical flows |
| P1-9 | Resident complaint form → ticket (manual source) |
| P1-10 | Bill estimator using `fn_tiered_charge` |
| P1-11 | Daily rollup `$merge` + TTL archival |
| P1-12 | Load test of ingestion with measured throughput |

### P2: stretch (only if ahead on Day 11)

Optional cloud deploy; zone choropleth map; Redis cache (only with measured need); email via a dev SMTP catcher; weather-driven demand annotation; a simple z-score/EWMA detector alongside the rules; vector search over complaint text (Atlas Vector Search); PDF invoice download; dark mode.

**Cut order if behind:** P2 entirely → P1-12 → P1-11 → P1-10 → P1-6 → P1-3 → P1-8 (keep one E2E) → P1-1 reduced to one scripted burst button. **Never cut:** RLS, triggers, procedures, CTE/recursive queries, Mongo RUD + aggregations, the billing bridge, CI, docs.

---

## F. The 14-day execution plan (Dhanush & Gagan)

> **In simple words.** Dhanush leads the PostgreSQL side (schema, money logic, security inside the database). Gagan leads the MongoDB side and the website (simulator, charts, screens). Every day each person reviews the other's pull request, so both can explain and modify every part in the viva. By the end of Day 3 a real resident record flows from PostgreSQL, through the API, to the screen, with MongoDB connected too.

**Assumptions.** About 6–7 focused hours per person per day (≈ 90 hours each). Daily 15-minute sync at start, 15-minute demo-to-each-other at end. "Cross-review" means the other person reads and approves the PR and can explain it.

**Ownership summary.**

| Area | Lead | Reviewer / backup |
|---|---|---|
| PostgreSQL schema, constraints, migrations, seed | Dhanush | Gagan |
| Procedures, functions, triggers, queries, CTEs | Dhanush | Gagan |
| Auth, sessions, DB roles, RLS | Dhanush | Gagan |
| MongoDB collections, validators, pipelines | Gagan | Dhanush |
| Simulator, anomaly detector, worker | Gagan | Dhanush |
| Next.js UI, charts, E2E | Gagan | Dhanush |
| API route handlers | shared (each for their domain) | other |
| CI, tooling | Gagan | Dhanush |
| Docs | shared | other |

---

### Day 1: Foundations and decisions

**Objective:** Repo, tooling and both databases running locally and in CI; decisions written down.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | Write `docs/requirements.md` (problem, objectives, rubric matrix from section B). Write `docs/technology-justification.md` v1. Draft ER diagram (Mermaid) and get Gagan's sign-off. Write `db/postgres/migrations/0001_extensions_roles.sql` (extensions, `migrator`, `app_login`, `resident_role`, `staff_role`, `admin_role`). Migration runner `scripts/migrate.ts`. | Create GitHub repo, protect `main`, issue templates, labels (P0/P1/P2). `create-next-app` (TS, App Router, Tailwind, ESLint), add Prettier, Vitest, shadcn/ui init. `docker-compose.yml` (postgres:17, mongo:8, init users). `src/env.ts` Zod env. `.github/workflows/ci.yml` (install, lint, typecheck, test, build with service containers). Pin all versions. |
| Files | `docs/*`, `db/postgres/migrations/0001_*.sql`, `scripts/migrate.ts` | `package.json`, `docker-compose.yml`, `.env.example`, `src/env.ts`, `.github/workflows/ci.yml`, `eslint/prettier configs` |
| DB deliverables | Roles exist; migrations table | Containers healthy |
| Tests | `npm run db:migrate` on fresh DB twice (idempotent runner) | CI runs on a PR and goes green (inspect the run) |
| Git | PR #1 "docs: requirements & ER v1"; PR #2 "chore: scaffold + CI" | |
| Done when | Both PRs merged, CI green on `main`, `docker compose up` + `npm run dev` works on **both** laptops | |
| Risks | Docker on Windows (use WSL2); version incompatibilities → step back one minor | |
| Hours | 6 | 7 |

### Day 2: The relational core

**Objective:** Complete DDL with every constraint; seed data; Mongo collections created.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | `0002_core_tables.sql`: all 15 tables from D.3 with constraints and indexes. `T-SQL-02` constraint-rejection tests (Vitest + `pg`) for each CHECK/UNIQUE/EXCLUDE. Begin `scripts/seed.ts` (zones, residents, properties, meters, cycles, tariffs, users with Argon2id). | `db/mongo/setup.ts`: create `meter_readings` time-series, other collections with `$jsonSchema`, indexes, TTL; Mongo users. `T-MDB-01/02` tests. `src/server/pg.ts` (pool + `withUser`), `src/server/mongo.ts` (client cache). Test DB isolation helper (`smartcity_test`, truncate between tests). |
| Files | `migrations/0002_*.sql`, `tests/db/constraints.test.ts`, `scripts/seed.ts` | `db/mongo/*`, `src/server/pg.ts`, `src/server/mongo.ts`, `tests/helpers/*` |
| DB deliverables | All tables + constraints; seeded city | All collections + validators + indexes |
| Tests | Constraint suite passes | Mongo setup tests pass, bad doc rejected |
| Git | PR #3 "db: core schema + constraints"; PR #4 "mongo: collections & validators" | |
| Done when | `npm run db:migrate && npm run db:seed` builds the whole city from zero; CI runs DB tests against service containers | |
| Risks | `EXCLUDE` needs `btree_gist` (in 0001). Validator on time-series may be rejected → Zod-only fallback, documented | |
| Hours | 7 | 7 |

### Day 3: First vertical slice (milestone M1)

**Objective:** A resident logs in and sees their own property, meters and latest reading, with data from **both** databases.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | Auth: `src/server/auth/password.ts`, `session.ts`, `POST /api/v1/auth/login`, `logout`, `GET /api/v1/me`, `requireRole()`. Basic RLS on `resident`, `property`, `utility_meter` (`0006_rls.sql` v1). `T-AUTH-01..03`, `T-RLS-01`. | Simulator core `src/sim/generator.ts` (seeded PRNG, per-utility profiles, cumulative registers) and backfill 7 days for seeded meters. `GET /api/v1/properties/:id/live` (latest 60 readings). Login page, resident layout, dashboard showing property card (SQL) + mini chart (Mongo). `middleware.ts` redirect. |
| Files | `src/server/auth/*`, `src/app/api/v1/auth/**`, `migrations/0006_rls.sql` | `src/sim/*`, `src/app/(resident)/dashboard/page.tsx`, `src/app/login/page.tsx`, `src/app/api/v1/properties/[id]/live/route.ts` |
| Tests | Wrong password → 401; resident A requesting B's property → 404 at API **and** 0 rows in SQL as `resident_role` | Generator determinism (same seed → identical docs); backfill count |
| Git | PR #5 "auth + RLS v1"; PR #6 "simulator core + resident dashboard v1" | |
| Done when | **M1 definition of done (section J.1) met**, demoed to each other | |
| Risks | RLS + pooling subtlety → always `SET LOCAL` inside a transaction | |
| Hours | 7 | 7 |

### Day 4: Roles everywhere and the staff/admin shells

**Objective:** RLS on every table; three role areas exist; tickets can be created manually.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | Finish RLS + GRANTs on all tables for 3 roles; `T-RLS-02..05` (staff zone scope, admin all, resident cannot UPDATE invoice, resident cannot read `app_user`). Triggers T1 + T2 (ticket transitions + history) with success/failure tests. | Admin and staff layouts, role-aware nav, 403 page. Ticket API: `POST /tickets` (resident complaint), `GET /tickets`, `POST /tickets/:id/transitions`, assignment. Mongo `maintenance_logs` create on ticket open; add-note (`$push`). Staff ticket list + detail page with timeline. |
| Tests | Illegal transition rejected; history rows written with user id | API tests: resident complaint, staff transition, resident cannot transition |
| Git | PR #7 “rls: all tables”; PR #8 “tickets v1 (SQL + Mongo logs)” | |
| Done when | Each role logs in and sees a different, working area; manual ticket flows OPEN→CLOSED with history | |
| Hours | 7 | 7 |

### Day 5: Live simulator and analytics

**Objective:** Readings flow continuously; charts are real.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | Analytical SQL: C1 (MoM with `LAG`), Q6 (zone outstanding), Q5 (resident statement). Views for the admin dashboard. `GET /admin/summary` SQL half. Seed: 3 closed historical cycles of `meter_reading` derived from backfill. | Worker `scripts/worker.ts`: live tick loop (`insertMany`, `ordered:false`), start/stop via `sim_control` doc + admin toggle, `ingestion_batches`. Pipelines 1, 2, 5 (hourly, daily zone, meter health). Admin dashboard: city KPIs, zone bars, trend line, meter health table. Polling every 5 s with SWR-style hook. |
| Tests | Queries return expected values on seed 42 (`T-SQL-04` part 1) | Pipeline tests with fixed fixtures; out-of-order + duplicate readings don’t change hourly units (`T-MDB-04`) |
| Git | PR #9 “sql analytics v1”; PR #10 “worker + analytics dashboard” | |
| Done when | Turn simulator on → admin chart moves within 10 s; turn off → meters show “last seen” ageing | |
| Risks | Chart performance with many points → server-side downsampling via `$dateTrunc` | |
| Hours | 7 | 7 |

### Day 6: Billing engine (SQL)

**Objective:** Correct tiered bills, atomically, with no duplicates.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | `fn_tiered_charge`, `sp_generate_invoice` (advisory lock, idempotent), trigger T3 (total sync + lock), T4 (tariff contiguity), T6 (reading lock). Billing unit tests incl. rounding (`ROUND(...,2)` half-up at line level, documented). Concurrency test: two parallel `CALL`s → one invoice. | Bridge pipeline 3 (cycle first/last register with `$top`/`$bottom`), `aggregation_run` lifecycle, upsert into `meter_reading` via `pg`. Admin “Billing” page: choose cycle → run aggregation → generate invoices → results table with errors. |
| Tests | `T-SQL-05` 450 kWh → ₹3,900.00; boundary units; no tariff → error; duplicate call → same invoice; overlapping tariff insert fails | `T-X-01` aggregate → readings → invoices end-to-end on test DBs; rerun run → no duplicates |
| Git | PR #11 “billing procedures + triggers”; PR #12 “aggregation bridge + billing UI” | |
| Done when | For the open cycle, admin clicks two buttons and every property has exactly one correct invoice; rerunning changes nothing | |
| Risks | Hardest day. If behind, Day 7 morning absorbs it; P1 work moves | |
| Hours | 8 | 7 |

### Day 7: Payments and resident billing UI

**Objective:** Residents see and pay bills safely.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | `sp_record_payment` with idempotency key, amount check, simulated FAIL path with SAVEPOINT; T5. `fn_mark_overdue` + late fee. Q3 (3+ overdue by zone). `POST /invoices/:id/payments`. | Resident invoices list + invoice detail (line items per slab, rate snapshot, due date, status), Pay dialog (“Simulated gateway” label, success/fail choice), payment history. Overdue banner. Measure latest-reading query latency (decide on Redis: expected *no*). |

| | Dhanush | Gagan |
|---|---|---|
| Tests | `T-SQL-06` double submit same key → one row; wrong amount; already paid; failure leaves PENDING | `T-API-PAY` 409 on replay; UI states |
| Git | PR #13 “payments + overdue”; PR #14 “resident billing UI” | |
| Done when | Mid-point demo: proofs 1–3 and 5–7 from section J work end to end | |
| Hours | 7 | 7 |

### Day 8: Anomaly detection

**Objective:** Explainable detection that opens one ticket per problem.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | `fn_open_anomaly_ticket` with partial-unique-index upsert, property `service_status` update, tests for duplicates. Q2 (correlated) and Q4 (NOT EXISTS). | Detector `src/server/anomaly/rules.ts`: **Spike** (hourly units > `k` × same-hour 14-day baseline and > absolute floor); **Sustained** (3 consecutive hours > 1.5×); **Leak** (water flow > 2 L/min continuously for 120 min including 01:00–05:00); **Missing data** (no reading for 15 min); **Register rollback** (malfunction). Each produces a `reason` string (“Flow never dropped below 2.4 L/min for 140 min; baseline night flow 0.1 L/min”). Outbox write to `anomaly_events`, worker sync loop. Admin alert feed with acknowledge (`$set`). |
| Tests | Same anomaly key twice → one ticket, occurrence_count = 2 | `T-ANOM-01..06`: each rule positive + negative fixture; duplicate events; crash between Mongo and SQL then retry (`T-X-02`) |
| Git | PR #15 “anomaly ticket fn + queries”; PR #16 “anomaly detector + alerts” | |
| Done when | Injecting a leak for one meter creates exactly one CRITICAL ticket with a readable reason, visible to admin and the zone’s staff | |
| Hours | 7 | 8 |

### Day 9: Disaster scenario and maintenance workflow polish

**Objective:** The showpiece scenario works reliably.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | Q7 (staff backlog), C2 (ageing), C3 (revenue by tier), Q1 (recursive zone roll-up, fixed to avoid double counting). Staff auto-assignment rule (least-loaded staff in zone) as SQL function. | Scenario page `/admin/scenarios`: “Pipe Burst in Ward W-3” and “Local Outage” buttons → seeded injection (idempotency key) → affected properties list, live tickets appearing, zone tile turning red. Notifications to affected residents (Mongo create). Resident ticket tracking + notifications panel (mark read, dismiss = **delete**). |
| Tests | `T-SQL-04` part 2 for all queries | `T-E2E-03` burst → ticket appears (Playwright) |
| Git | PR #17 “sql queries complete”; PR #18 “disaster scenarios + notifications” | |
| Done when | The burst demo runs 5 times in a row from `demo:reset` with identical results | |
| Hours | 7 | 7 |

### Day 10: SQL Lab, reconciliation, test hardening

**Objective:** Make the database work *visible*; close test gaps.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | Query catalogue files `db/postgres/queries/*.sql` with comments; `GET /admin/sql-lab/:queryId` runs whitelisted queries only (no free-form SQL). Constraint-violation demo set (“Try invalid data” buttons that show the real Postgres error). Reconciliation SQL side. | SQL Lab UI (query text + result table + explanation). Mongo Lab panel showing pipeline JSON + output for pipelines 1–5. Reconciliation page. Fill test gaps from coverage report; negative tests for malformed/missing/out-of-order readings (`T-MDB-05..07`). |
| Tests | Full `npm test` locally and in CI; coverage of `src/server` ≥ 70% | |
| Git | PR #19 “sql lab + constraint demos”; PR #20 “mongo lab + reconciliation + tests” | |
| Done when | Every row of the traceability matrix has a passing test ID and a demo step | |
| Hours | 7 | 7 |

### Day 11: Feature freeze, E2E, security review

**Objective:** Stop adding features; prove quality.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | Security pass: injection attempts on every input, IDOR checks via direct URLs, session expiry/lockout, `npm audit`, secrets scan, headers (CSP basics). Load test ingestion (`scripts/loadtest.ts`) and record real numbers. | Playwright E2E `T-E2E-01..04` (resident isolation, pay invoice, staff resolves ticket, burst scenario) in CI. UI states: loading skeletons, empty states, error toasts, confirmation dialogs, accessibility pass (labels, focus, contrast). |
| Git | PR #21 “security hardening”; PR #22 “e2e + ux states” | |
| Done when | **Feature freeze** at end of day. All P0 green. Only bug fixes after this. | |
| Hours | 7 | 7 |

### Day 12: Documentation and deployment

**Objective:** Everything an examiner may ask for exists in `docs/`.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | `docs/relational-schema.md`, ER PNG export, `docs/sql-catalogue.md` (each object: purpose, SQL, when it runs, tests, viva line), testing report from real CI output. | README (setup, scripts, screenshots), `docs/architecture.md` + data-flow diagram, `docs/mongo-design.md`, `docs/api.md`. Backup laptop setup. Optional cloud deploy (P2) only if everything is green. |
| Git | PR #23 “docs: database”; PR #24 “docs: app + setup” | |
| Done when | A classmate can clone and run the project from the README alone (test it) | |
| Hours | 6 | 6 |

### Day 13: Rehearsal and modification drill

**Objective:** Both members can demo and *change* anything.

| | Dhanush | Gagan |
|---|---|---|
| Tasks | Run the full demo script twice (timed, 12 min). **Modification drill:** Gagan asks Dhanush for 3 live changes in Dhanush’s *non-owned* area (e.g. add a chart filter, add a field to a Mongo validator, change refresh rate). Fix bugs found. Write `docs/viva.md` Q&A (DB side). | Run the demo twice. Drill: Dhanush asks Gagan for 3 live changes in SQL (e.g. add a tariff tier, add a CHECK, add a column to Q3). Fix bugs. Write viva Q&A (app/NoSQL side). `docs/contributions.md` from Git history. |
| Done when | Each person completes their 3 changes in under 10 minutes each with tests passing | |
| Hours | 6 | 6 |

### Day 14: Buffer, final verification, freeze

**Objective:** Nothing new. Verify and tag.

| | Both |
|---|---|
| Tasks | Morning: buffer for anything that slipped. Afternoon: fresh clone on both laptops → migrate → seed → full test suite → build → full demo. Inspect the final CI run on `main`. Update the traceability matrix statuses with evidence links. Tag `v1.0.0`. Print the one-page demo checklist. |
| Done when | Section J checklist fully ticked with evidence |
| Hours | 4–6 each |

**Effort check.** Dhanush ≈ 93 h, Gagan ≈ 95 h. The plan front-loads integration: both databases are connected by Day 2 and a full SQL+Mongo vertical slice exists on Day 3. Billing (Day 6) and anomaly (Day 8) are the two highest-risk days and each has the following day as overflow.

---

## G. Repository and file structure

```
smart-city-utilities/
├─ README.md
├─ .env.example                 # placeholders only
├─ docker-compose.yml           # postgres:17, mongo:8 (+ init scripts)
├─ package.json                 # scripts: dev, build, worker, db:migrate, db:seed, demo:reset, test, test:e2e, reconcile
├─ .github/
│  ├─ workflows/ci.yml          # lint, typecheck, unit+integration (service containers), build, e2e
│  ├─ pull_request_template.md
│  └─ ISSUE_TEMPLATE/
├─ db/
│  ├─ postgres/
│  │  ├─ migrations/
│  │  │  ├─ 0001_extensions_roles.sql
│  │  │  ├─ 0002_core_tables.sql
│  │  │  ├─ 0003_functions_procedures.sql
│  │  │  ├─ 0004_triggers.sql
│  │  │  ├─ 0005_views.sql
│  │  │  └─ 0006_rls_grants.sql
│  │  └─ queries/               # q1_zone_rollup.sql … c3_revenue_by_tier.sql, dml.sql (catalogue)
│  └─ mongo/
│     ├─ setup.ts               # collections, validators, indexes, TTL, users
│     ├─ schemas/*.json         # $jsonSchema per collection
```

```
│     └─ pipelines/*.ts          # hourly, dailyZone, cycleTotals, baseline, meterHealth
├─ scripts/
│  ├─ migrate.ts  seed.ts  demo-reset.ts  worker.ts  reconcile.ts  loadtest.ts
├─ src/
│  ├─ env.ts
│  ├─ middleware.ts              # UX redirect only
│  ├─ app/
│  │  ├─ login/page.tsx
│  │  ├─ (resident)/dashboard, invoices, invoices/[id], tickets, notifications
│  │  ├─ (staff)/tickets, tickets/[id]
│  │  ├─ (admin)/overview, zones/[id], properties/[id], billing, scenarios, alerts, sql-lab, reconciliation
│  │  └─ api/v1/
│  │     ├─ auth/{login,logout}/route.ts   me/route.ts
│  │     ├─ properties/[id]/{route.ts,live/route.ts}
│  │     ├─ invoices/{route.ts,[id]/route.ts,[id]/payments/route.ts}
│  │     ├─ tickets/{route.ts,[id]/route.ts,[id]/transitions/route.ts,[id]/notes/route.ts}
│  │     ├─ admin/{summary,zones,billing/runs,billing/invoices,scenarios,alerts,sql-lab/[queryId],reconcile}/route.ts
│  │     └─ notifications/{route.ts,[id]/route.ts}
│  ├─ components/                # ui/ (shadcn), charts/, tables/, layout/
│  ├─ lib/                       # client helpers, formatters (money as string), fetcher
│  ├─ sim/                       # prng.ts, profiles.ts, generator.ts, scenarios.ts
│  └─ server/
│     ├─ pg.ts  mongo.ts  errors.ts
│     ├─ auth/                   # password.ts, session.ts, requireRole.ts
│     ├─ repos/                  # SQL access per aggregate (parameterised only)
│     ├─ services/               # billing.ts, payments.ts, tickets.ts, analytics.ts, reconcile.ts
│     ├─ anomaly/                # rules.ts, detector.ts, outbox.ts
│     └─ schemas/                # Zod schemas shared with client forms
├─ tests/
│  ├─ unit/  db/  mongo/  api/  cross-db/  helpers/
│  └─ e2e/                       # Playwright specs
└─ docs/
   ├─ requirements.md  traceability.md  technology-justification.md
   ├─ architecture.md  er-diagram.md  relational-schema.md  mongo-design.md
   ├─ sql-catalogue.md  api.md  testing-report.md  demo-script.md
   ├─ contributions.md  viva.md  known-limitations.md
```

# H. Testing, security, Git and CI strategy

## H.1 Testing pyramid

> **In simple words.** Lots of small fast tests for the maths (billing, anomaly rules), a good number of tests that talk to real databases, and a handful of full browser tests for the most important journeys. Tests use their own databases so they can never damage the demo data.

| Level | Tool | What | Examples (IDs) |
|---|---|---|---|
| Unit | Vitest | Pure logic | Generator determinism; anomaly rules (`T-ANOM-*`); money formatting; Zod schemas |
| PostgreSQL integration | Vitest + real Postgres (`smartcity_test`) | Constraints, functions, procedures, triggers, queries, RLS | `T-SQL-01..09`, `T-RLS-01..05` |
| MongoDB integration | Vitest + real Mongo (`smartcity_test`) | Collections, validators, pipelines, RUD | `T-MDB-01..08` |
| API | Vitest calling route handlers | Status codes, auth, validation, envelopes | `T-API-*`, `T-AUTH-01..06` |
| Cross-DB | Vitest with both DBs | Bridge idempotency, outbox retry, reconciliation | `T-X-01..03` |
| E2E | Playwright | 4 critical journeys | `T-E2E-01..04` |

**Negative cases required (each maps to at least one test):** invalid reading (negative register, wrong utility fields) → 400; missing fields → 400 + validator rejection; out-of-order readings → same totals; duplicate readings → same totals; unauthorised user → 401/403/404; cross-resident access → 404 + 0 rows in SQL; duplicate invoice generation → one invoice; repeated payment request → one row, 409 or original result; failed DB write (forced via a CHECK violation mid-procedure) → full rollback; sensor outage → MISSING_DATA anomaly; duplicate anomaly events → one ticket; invalid tariff (overlap, gap, unbounded middle tier) → rejected.

**Test data isolation.** Separate `smartcity_test` databases in both engines; each test file runs in a transaction that rolls back (Postgres) or uses a fresh collection prefix (Mongo); CI uses throwaway service containers. The demo dataset is only touched by `demo:reset`.

## H.2 Security checklist

| Threat | Control |
|---|---|
| Cross-resident access (IDOR) | Ownership check in services **and** RLS; 404 for not-yours |
| Privilege escalation | Role comes from DB session row, never from the client; `SET LOCAL ROLE` from server-side mapping; admin routes double-checked |
| SQL injection | Parameterised `pg` queries only; SQL Lab runs whitelisted files, never user SQL; lint rule against template-string SQL |
| Mongo operator injection | Filters built from Zod-parsed primitives; reject keys starting with `$` |
| Exposed credentials | `.env` git-ignored; `.env.example` placeholders; secret scanning on GitHub; separate least-privilege DB users |
| Weak sessions | Random 256-bit tokens, hashed at rest, HttpOnly/Secure/SameSite, expiry, revocation, lockout |
| Unvalidated input | Zod on every route; DB constraints as the final net |
| File uploads | None in P0/P1 (attachments are metadata only). If added, size/type limits and no execution |

| Threat | Control |
|---|---|
| Unauthorised admin actions | `requireRole('ADMIN')` + `admin_role` GRANTs; audit entries in history/logs |
| Replayed payments | Idempotency key UNIQUE + one-SUCCEEDED partial index |
| Real personal data | Synthetic names, `SYN-` refs, `example.test` emails, UI label “Synthetic demo data” |

### H.3 Git workflow (two people)

- `main` is protected: PR required, 1 approval (the other person), CI checks required (`lint`, `typecheck`, `test`, `build`), no force-push, linear history (squash merge).
- Short-lived branches: `feat/billing-procedure`, `fix/rls-invoice`, `docs/sql-catalogue`; one day or less each.
- Conventional commits: `feat(db): add sp_generate_invoice with advisory lock`.
- PR template: what changed, rubric rows affected, how tested (commands + output), screenshots, checklist (tests added, docs updated, no secrets).
- Issues per backlog item with P0/P1/P2 labels and the rubric row.
- Each person authors their own commits from their own GitHub account, which is the evidence for row 14 (individual contribution). Pairing sessions use `Co-authored-by:` trailers.

### H.4 CI (GitHub Actions)

Jobs: `lint` (ESLint + Prettier check), `typecheck` (`tsc --noEmit`), `test` (Vitest with `postgres:17` and `mongo:8` service containers, runs migrations + Mongo setup first), `build` (`next build`), `e2e` (Playwright on the built app, on PRs to `main`). Cache npm. Upload test and coverage reports as artifacts; the testing report in `docs/` quotes the actual run, never an assumed one.

---

## I. Risks, assumptions and contingencies

| Risk | Likelihood | Impact | Mitigation / contingency |
|---|---|---|---|
| RLS + connection pooling leaks identity | Medium | Critical (security marks) | `SET LOCAL` inside transactions only; test that runs two users back-to-back on one pooled connection |
| Billing correctness (rounding, boundaries) | Medium | High | `NUMERIC` only, never JS floats; table-driven tests with hand-computed expectations |
| Time-series collection limits (validators, updates/deletes) | Medium | Medium | RUD marks come from `anomaly_events`, `maintenance_logs`, `notifications`; readings validated by Zod; documented |
| Simulator overloads laptop | Low | Medium | Default 300 meters at 1 doc/5 s; backfill uses 15-min granularity |
| Day 6 or Day 8 overrun | High | Medium | Next day is overflow; P1 items move; cut order in section E |
| Next.js / library version breakage | Medium | Medium | Pin on Day 1, step back one minor if broken, no upgrades after Day 5 |
| One member unavailable for a day | Medium | High | Cross-review means the other can continue; issues are small |
| Wi-Fi or cloud failure on viva day | Medium | Critical | Local Docker demo is primary; backup laptop; `demo:reset` |
| Examiner asks for live modification | High | High | Day 13 drill; small, well-named modules; SQL in plain files |
| Over-claiming | Low | High (credibility) | Matrix only marks “Done” with a passing test link; measured numbers only |

**Assumptions.** Viva can be run from a laptop; both members have Docker-capable machines; local PostgreSQL superuser is available (needed for roles/RLS); synthetic data is acceptable; the rubric PDF in the project is the current one.

---

## J. Final acceptance checklist (demo and viva)

### J.1 The single best first milestone: M1 “Hybrid vertical slice” (end of Day 3)

**Why this one:** it proves the riskiest architectural ideas at once (both databases connected, real login, database-level RBAC, the SQL + NoSQL merge on one screen) before any feature work depends on them.

**Definition of done (all must be true, with evidence):**

1. `docker compose up -d && npm run db:migrate && npm run db:seed -- --seed 42` builds both databases from scratch on **both** laptops.
2. Resident `asha@example.test` logs in with a password verified by Argon2id; a session row exists in `user_session`; cookie is HttpOnly.
3. `/dashboard` shows Asha’s property code, address and zone (PostgreSQL) and a chart of her electricity meter’s last 60 readings (MongoDB), served by one API call that queries both.
4. Requesting another resident’s property via `/api/v1/properties/<other-id>` returns 404.
5. In `psql`, `SET ROLE resident_role; SET app.user_id = '<asha>'; SELECT count(*) FROM property;` returns only Asha’s properties.
6. Running the simulator generator twice with seed 42 produces byte-identical documents.
7. CI on `main` is green (lint, typecheck, tests, build), checked by opening the run.
8. PRs #1–#6 are merged with one review each; both members can explain every file in the slice.

### J.2 Demo script (12 minutes, from `npm run demo:reset`)

| # | Step | What the examiner sees | Proof |
|---|---|---|---|
| 1 | Log in as Resident A, then try Resident B's invoice URL | 404; then `psql` as `resident_role` returns 0 rows | Isolation |
| 2 | Start simulator; open Mongo Compass on `meter_readings` | New documents with different fields per utility | IoT → Mongo |
| 3 | Admin overview | Charts and zone totals move | Analytics |
| 4 | Scenarios → Pipe Burst in Ward W-3 | Alert with reason, one CRITICAL ticket, zone turns red; second burst bumps occurrence count | Anomaly → ticket |
| 5 | Billing → Run aggregation for cycle | `meter_reading` rows with opening/closing registers | Usage → SQL |
| 6 | Generate invoices; open Asha's bill | Slab lines 100×5 + 200×8 + 150×12 = ₹3,900.00; rerun creates nothing new | Tiered billing |
| 7 | Asha pays (simulated); click Pay twice; try a failing payment | One SUCCEEDED row, invoice PAID; failed attempt recorded, invoice unchanged | Safe payment |
| 8 | SQL Lab: Q1 recursive, Q2 correlated, Q3 overdue, C1 MoM | SQL text + results | Complex SQL |
| 9 | "Try invalid data": overlapping tariff, month 13, duplicate invoice, illegal ticket transition | Real Postgres errors | Constraints/triggers |
| 10 | GitHub: PR list, CI run, test report | Green checks, both contributors | Quality |

### J.3 Final checklist

- ☐ All 14 rubric rows have a passing test ID and a demo step in `docs/traceability.md`
- ☐ ER diagram (PNG + Mermaid) and relational schema doc match the migrations
- ☐ DDL migrations run cleanly on an empty database
- ☐ Mongo collections, validators, indexes and TTL exist; RUD shown
- ☐ Technology justification written (1 mark, no partial credit)
- ☐ 5 procedures/functions, 6 triggers, 10 queries/CTEs each with success + failure test
- ☐ Three roles, RLS on every user-data table, `psql` proof rehearsed
- ☐ Billing ₹3,900 example reproduces exactly; duplicate and replay safety shown
- ☐ Pipe Burst scenario works 5/5 times after reset
- ☐ CI green on the final `main` commit (inspected, linked)
- ☐ README setup tested by someone else
- ☐ Known limitations documented (simulated payments, synthetic data, measured throughput only)
- ☐ `docs/contributions.md` and Git history show both members' work
- ☐ Each member completed the modification drill
- ☐ `docs/viva.md` rehearsed; backup laptop ready

### J.4 Viva quick answers (starter set; full list in `docs/viva.md`)

- **Why two databases?** Money needs ACID transactions, constraints and joins (PostgreSQL). Sensor data is huge, append-only, time-ordered and differently shaped per meter type (MongoDB time-series).
- **How do they stay consistent without a shared transaction?** Idempotent hand-offs: unique keys in SQL, deterministic IDs in Mongo, an outbox flag for unsynced events, and a reconciliation report.
- **Why cumulative registers?** Duplicated or late readings cannot change the bill; consumption is closing − opening.
- **Where is access control enforced?** In three places; only the database layer cannot be bypassed by a bug in our code (RLS with `app.user_id`).
- **What stops a double bill or double payment?** `UNIQUE(property_id, cycle_id)`, an advisory lock, `idempotency_key UNIQUE`, and a partial unique index allowing one successful payment.
- **Why a partial unique index for tickets?** One *active* ticket per anomaly key, but history can hold many resolved ones.
- **What would you do at real scale?** Partition invoices by cycle, shard readings by meter, add read replicas, move the worker to a queue; Redis only if measured latency demands it.

---

## Phase 3: what needs Dhanush's approval

1. Keep the 10 source tables and add 5 (`zone`, `aggregation_run`, `ticket_status_history`, `app_user`, `user_session`); modify `payment` to store attempts and `utility_rate` for open-ended slabs.
2. Tickets live in PostgreSQL; notes and logs in MongoDB.
3. Next.js only (no Express); raw SQL with `pg` (no ORM); Mongo driver without Mongoose.
4. Cut Redis, the weather API and real email/SMS; payments are simulated and labelled.
5. Local Docker demo as the primary viva setup.
6. Ownership split: Dhanush leads PostgreSQL + auth/RLS, Gagan leads MongoDB + simulator + UI.

Once approved, Phase 4 starts with Day 1 and milestone M1.